TDA를 활용한 류이치 사카모토의 〈hibari〉 구조 분석 및 위상구조 기반 AI 작곡 파이프라인의 제시

저자: 김민주
지도: 정재훈 (KIAS 초학제 독립연구단), Claude
작성일: 2026.04.20
키워드: Topological Data Analysis, Persistent Homology, Tonnetz, Discrete Fourier Transform, Music Generation, Vietoris-Rips Complex, Jensen-Shannon Divergence

---

초록 (Abstract)

본 연구는 사카모토 류이치의 2009년 앨범 out of noise 수록곡 "hibari"를 대상으로, 음악의 구조를 **위상수학적으로 분석**하고 그 위상 구조를 **보존하면서 새로운 음악을 생성**하는 파이프라인을 제안한다. 전체 과정은 네 단계로 구성된다. (1) MIDI 전처리: 두 악기를 분리하고 8분음표 단위로 양자화. (2) Persistent Homology: 네 가지 거리 함수(frequency, Tonnetz, voice leading, DFT)로 note 간 거리 행렬을 구성한 뒤 H_1 cycle을 추출. (3) 중첩행렬 구축: cycle의 시간별 활성화를 이진 또는 연속값 행렬로 기록. (4) 음악 생성: 중첩행렬을 seed로 하여 확률적 샘플링 기반의 Algorithm 1과 FC / LSTM / Transformer 신경망 기반의 Algorithm 2 두 방식으로 음악 생성.

$N=20$회 통계적 반복을 통한 정량 검증에서, **Algorithm 1**(확률적 샘플링) 기반으로 DFT 거리 함수가 네 거리 함수 중 최우수로 확인되었다 — frequency 대비 pitch JS divergence를 0.0344 ± 0.0023에서 0.0213 ± 0.0021로 **약** 38.2% **감소**시켰다 ($p < 10^{-20}$). 이후 DFT 기반 continuous OM에서 $\alpha = 0.25$ ($K=14$) 조건의 per-cycle τ_c 최적화를 적용해 Algorithm 1 신규 최저 0.01156 ± 0.00147 ($N=20$)를 달성했다. 이는 직전 $\alpha = 0.5$ 기준 (0.01489) 대비 추가 -22.35% 개선이며 (Welch $p = 4.94 \times 10^{-11}$), $\alpha = 0.25$가 §5.7 binary OM과 per-cycle τ_c 양쪽 모두에서 최적임이 이중으로 확인되었다. **Algorithm 2**에서는 연속값 중첩행렬 입력의 FC가 0.00035 ± 0.00015 ($N=10$)로 최우수였고, Transformer 대비 Welch $p = 1.66 \times 10^{-4}$로 유의 우위를 보였다. 두 최저값은 이론 최댓값 $\log 2 \approx 0.693$의 각각 약 1.67% (Algo1)와 0.05% (Algo2)다.

본 연구의 intra / inter / simul 세 갈래 가중치 분리 설계는 hibari의 두 악기 구조 — inst 1은 쉼 없이 연속 연주, inst 2는 모듈마다 규칙적 쉼을 두며 겹쳐 배치 — 를 수학적 구조에 반영한 것이며, 두 악기의 활성/쉼 패턴 관측 (inst 1 쉼 0개, inst 2 쉼 64개) 이 이 설계를 경험적으로 정당화한다.

---

1. 서론 — 연구 배경과 동기

1.1 연구 질문

음악은 시간 위에 흐르는 소리들의 집합이지만, 그 구조는 단순한 시간 순서만으로 포착되지 않는다. 같은 모티브가 여러 번 반복되고, 서로 다른 선율이 같은 화성 기반 위에서 엮이며, 전혀 관계없어 보이는 두 음이 같은 조성 체계 안에서 등가적 역할을 한다. 이러한 층위의 구조를 수학적으로 포착하려면 "어떤 두 대상이 같다(혹은 가깝다)"를 정의하는 **거리 함수**와, 그로부터 파생되는 **위상 구조**를 다루는 도구가 필요하다.

본 연구는 다음의 세 가지 질문에서 출발한다.

1. **위상 구조를 "보존한 채" 새로운 음악을 생성할 수 있는가?** 보존의 기준은 무엇이며, 보존 정도를 어떻게 정량적으로 측정하는가?
2. **거리 함수의 선택이 실제로 생성 품질에 유의미한 영향을 주는가?** 단순 빈도 기반 거리 대신 음악 이론적 거리 (Tonnetz, voice leading, DFT)를 사용하면 얼마나 나은가?
3. **위상 구조를 보존한 음악이 실제로 아름답게 들리는가?** 수학적으로 유사한 위상 구조를 가지도록 생성된 음악이 청각적으로도 원곡의 미학적 인상을 전달하는가? 본 보고서 말미에 첨부된 QR코드를 통해 생성된 음악을 직접 감상할 수 있다.

1.2 연구 대상 — 왜 hibari인가

본 연구의 대상곡은 사카모토 류이치의 out of noise (2009) 수록곡 "hibari" 이다. 이 곡을 선택한 이유는 다음과 같다.

선행연구의 확장에 적합. 단선율의 국악에 TDA를 적용한 선행연구(정재훈 외, 2024)를 화성음악으로 확장함에 있어, hibari는 복잡성을 내포하면서도 규칙적인 모듈 구조로 일정한 패턴이 있어 모델링이 용이하였다.

미학적 특수성. out of noise 앨범은 "소음과 음악의 경계"를 탐구하는 실험적 작업이며, hibari는 전통적 선율 진행이 아니라 음들의 공간적 배치에 가까운 방식으로 구성된다. 이 특성은 본 연구의 실험 결과 (§4.3)에서 DL 모델 선택과 직접적으로 공명한다.

---

2. 수학적 배경

본 절에서는 본 연구의 파이프라인을 이해하기 위해 필요한 수학적 도구들을 정의하고, 각 도구가 음악 구조 분석에서 어떻게 사용되는지를 서술한다.

2.1 Vietoris-Rips Complex

정의 2.1. 거리 공간 (X, d)와 양의 실수 $\varepsilon > 0$이 주어졌을 때, **Vietoris-Rips complex** $\mathrm{VR}_\varepsilon(X)$는 다음과 같이 정의되는 복합체(simplicial complex)이다:

$$\mathrm{VR}_\varepsilon(X) = \left\{\sigma \subseteq X \,|\, \forall x_i, x_j \in \sigma,\ d(x_i, x_j) \leq \varepsilon\right\}$$

즉, 점 집합 X의 부분집합 σ에 속한 **모든 점 쌍 사이의 거리가** ε **이하**이면 σ를 심플렉스(simplex)로 포함시킨다.

구성 요소:

0-simplex (vertex): 각 점 $x_i \in X$. 단일 점은 거리 조건이 없으므로 어떤 ε에서도 포함된다.

1-simplex (edge): $d(x_i, x_j) \leq \varepsilon$인 두 점의 쌍 $\{x_i, x_j\}$

2-simplex (triangle): 세 점이 서로 모두 ε 이내인 부분집합

Filtration 구조와 포함관계. ε 값을 0부터 연속적으로 키우면, 점 집합 X 자체는 변하지 않은 채 **새로운 심플렉스만 점차 추가된다.** $\varepsilon = 0$일 때 $\mathrm{VR}_0(X)$는 각 점만을 0-simplex로 포함하는 이산적인 점 집합(discrete set)이다 — 아직 어떤 edge도 없으므로 이것은 X 그 자체와 같다. ε이 커지면서 두 점 사이 거리가 ε 임계를 처음 넘는 순간에 1-simplex(edge)가 추가되고, 세 점이 모두 ε 이내가 되면 2-simplex(삼각형)가 추가된다. 즉 $\varepsilon_1 < \varepsilon_2$이면 $\mathrm{VR}_{\varepsilon_1}(X)$의 모든 심플렉스가 $\mathrm{VR}_{\varepsilon_2}(X)$에도 그대로 들어 있다. 따라서 다음의 포함관계는 항상 성립한다:

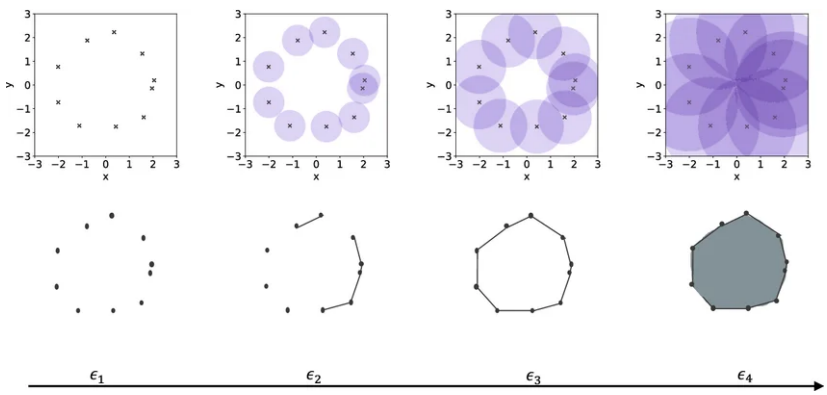
$$\mathrm{VR}_0(X) \subseteq \mathrm{VR}_{\varepsilon_1}(X) \subseteq \mathrm{VR}_{\varepsilon_2}(X) \subseteq \cdots \subseteq \mathrm{VR}_{\varepsilon_n}(X)$$

표기 편의를 위해 $K_i := \mathrm{VR}_{\varepsilon_i}(X)$로 두면:

$$K_0 \subseteq K_1 \subseteq K_2 \subseteq \cdots \subseteq K_n$$

이를 **filtration**이라 부르며, 변화가 일어나는 임계값 ε_i들이 곧 심플렉스의 birth/death 시점이 된다.

본 연구에서의 사용: $X = \{n_1, n_2, \ldots, n_{23}\}$은 hibari에 등장하는 23개의 고유 note이며, $d(n_i, n_j)$는 두 note 간 거리이다.



2.2 Simplicial Homology

정의 2.2. Simplex complex K에 대해 n차 호몰로지 군(homology group) $H_n(K)$는 K 안에 존재하는 n차원 "구멍"의 대수적 표현이다. 직관적으로:

$H_0(K)$: 연결 성분(connected components)의 수

$H_1(K)$: 1차원 cycle의 수 (닫힌 고리 모양으로 둘러싸인 영역)

$H_2(K)$: 2차원 빈 공간(void)의 수 (3차원 공동을 둘러싼 표면)

$H_n(K)$는 아벨 군이며, $\mathrm{rank}(H_n(K))$ = **Betti number** β_n은 서로 독립적인 n차원 구멍의 개수를 나타낸다.

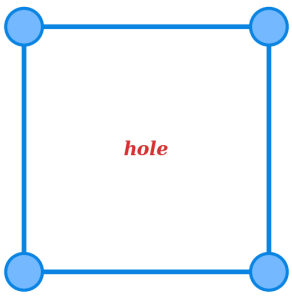
Boundary 연산자와 호몰로지 — 선형대수적 계산 예시. H_n이 어떻게 계산되는지를 간단한 예시로 보인다. 점 4개 $\{a, b, c, d\}$와 edge $\{ab, bc, cd, da, ac\}$, 삼각형 $\{abc\}$로 이루어진 복합체 K를 생각하자.

boundary 연산자 ∂_1 (edge → vertex)과 ∂_2 (삼각형 → edge)를 $\mathbb{F}_2$ (mod 2) 위에서 행렬로 표현하면:

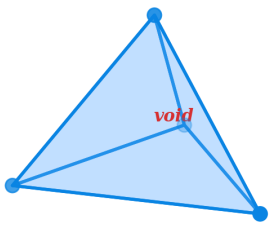
```
∂₁ (edge → vertex):          ∂₂ (triangle → edge):
      ab bc cd da ac                 abc
a [   1  0  0  1  1 ]          ab [   1 ]
b [   1  1  0  0  1 ]          bc [   1 ]
c [   0  1  1  0  0 ]          cd [   0 ]
d [   0  0  1  1  0 ]          da [   0 ]
                               ac [   1 ]
```

$H_1(K) = \ker(\partial_1)/\mathrm{im}(\partial_2)$이다. $\ker(\partial_1)$은 "닫힌 edge 체인들"의 공간이고, $\mathrm{im}(\partial_2)$는 "삼각형의 경계인 edge 체인들"의 공간이다. 이 예시에서 $\ker(\partial_1)$의 차원은 2 (두 개의 독립적인 닫힌 고리: $ab + bc + ca$와 $ac + cd + da$), $\mathrm{im}(\partial_2)$의 차원은 1 (abc의 경계 $= ab + bc + ca$). 따라서 $\beta_1 = 2 - 1 = 1$, 즉 독립적인 cycle이 1개이다. 직관적으로, 삼각형 abc가 한 cycle을 "채워서" 없앴고, 남은 사각형 $a - c - d - a$에 해당하는 cycle 1개가 살아남는다.

(a) 1D cycle: generator of H_1
4 points forming a square boundary,
no 2-simplex inside



(b) 2D void: generator of H_2
4 points forming a tetrahedron boundary,
no 3-simplex inside



본 연구에서의 사용: 본 연구는 주로 H_1 (1차 호몰로지)을 다룬다. 이는 음악 네트워크에서 서로 가까운 note들이 만드는 닫힌 cycle, 즉 순환적으로 연결된 note 그룹을 포착한다. 발견된 각 cycle은 곡의 구조적 반복 단위로 해석된다.

2.3 Persistent Homology

Filtration $K_0 \subseteq K_1 \subseteq \cdots \subseteq K_n$에서, 각 단계마다 H_1의 cycle 구성이 달라진다. **Persistent homology**는 이 과정에서 각 cycle이 어느 ε_i에서 처음 나타나고(birth) 어느 ε_j에서 사라지는지(death)를 추적한다.

Birth와 death의 음악적 의미:

Birth b: 거리 임계값 ε가 충분히 커져서 새로운 cycle이 형성되는 순간. 음악적으로는 "이 거리 척도에서 처음으로 닫힌 반복 구조가 발견되는 시점".

Death d: 더 큰 ε에서 그 cycle이 다른 cycle들의 합(정확히는 boundary)으로 표현될 수 있게 되어, 호몰로지 군 안에서 더 이상 독립적인 generator가 아니게 되는 순간. 음악적으로는 "거리 척도가 너무 느슨해져서 이 반복 구조가 다른 구조에 흡수되는 시점".

각 cycle은 (b, d) 쌍과, 그 cycle을 구성하는 vertex/edge 집합인 **cycle representative**가 함께 기록된다. 본 연구에서는 birth-death 쌍은 cycle의 "수명"을 측정하는 데, cycle representative는 어떤 note들이 그 cycle을 이루는지 식별하는 데 사용한다. 이 정보를 모은 것이 곡의 **위상적 지문(topological signature)**이다.

Persistence: $\mathrm{pers}(\mathrm{cycle}) = d - b$. 큰 persistence를 갖는 cycle은 다양한 거리 척도에서 살아남으므로 **위상적으로 안정한 구조**이며, 작은 persistence는 일시적이거나 노이즈에 가까운 구조이다.

본 연구에서의 사용: 거리 행렬 $D \in \mathbb{R}^{23 \times 23}$로부터 Vietoris-Rips filtration을 구성하고, 각 rate parameter r (가중치 비율, 후술)에서의 H_1 persistence를 계산한다. 발견된 모든 (b, d) 쌍과 cycle representative가 함께 cycle 집합을 정의하며, 이 cycle들이 다음 절의 중첩행렬 구축에 사용된다.

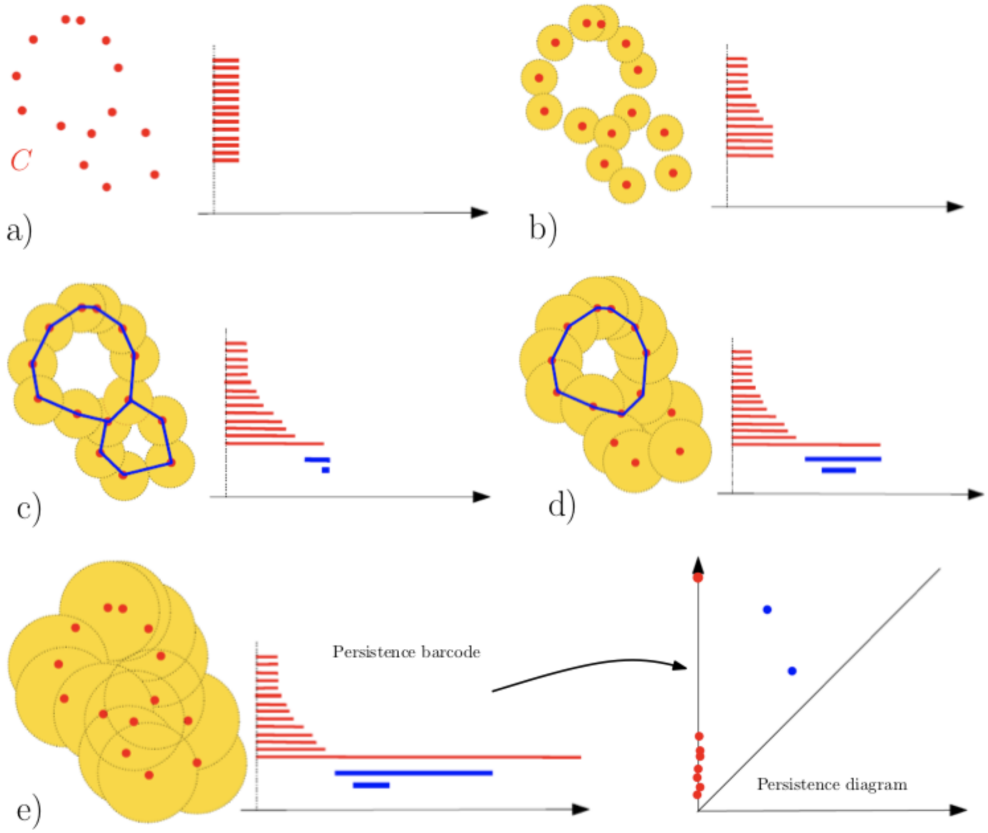


그림 2.3. (a)~(e)의 Persistence Barcode에서는 filtration value(ε)에 따라 발견되는 simplex를 막대 그래프를 통해 표시한다. 빨간색은 0-simplex(연결성분), 파란색은 1-simplex(cycle)를 표시하며 마지막 그림에서 바코드가 x축을 birth로 y축을 death로 하는 Persistence Diagram으로 표현된다.

2.4 빈도 기반 거리와 음악적 거리 함수

빈도 기반 거리. 본 연구의 기준 거리 d_{freq}는 두 note의 인접도(adjacency)의 역수로 정의된다. 인접도 $w(n_i, n_j)$는 곡 안에서 note n_i와 n_j가 시간적으로 연달아 등장한 횟수이다:

$$w(n_i, n_j) = \#\{\, t : n_i \text{ at time } t \text{ and } n_j \text{ at time } t+1 \,\}$$

거리는 $d_{\mathrm{freq}}(n_i, n_j) = 1/w(n_i, n_j)$로 정의되며 인접도가 0인 경우는 도달 불가능한 큰 값으로 처리한다.

정의 2.4. Tonnetz는 pitch class 집합 $\mathbb{Z}/12\mathbb{Z}$를 평면 격자에 배치한 구조이다. 여기서 **pitch class**는 옥타브 차이를 무시한 음의 동치류(equivalence class)로, 예컨대 C4, C5, C3는 모두 같은 pitch class "C"에 속한다.

Tonnetz의 격자 구조. pitch class $p \in \mathbb{Z}/12$를 좌표 (x, y)에 배치하되, 다음 관계를 만족시킨다:

가로 이동 (+1 in x): 완전5도 (perfect fifth, +7 semitones)

대각선 이동 (+1 in y): 장3도 (major third, +4 semitones)

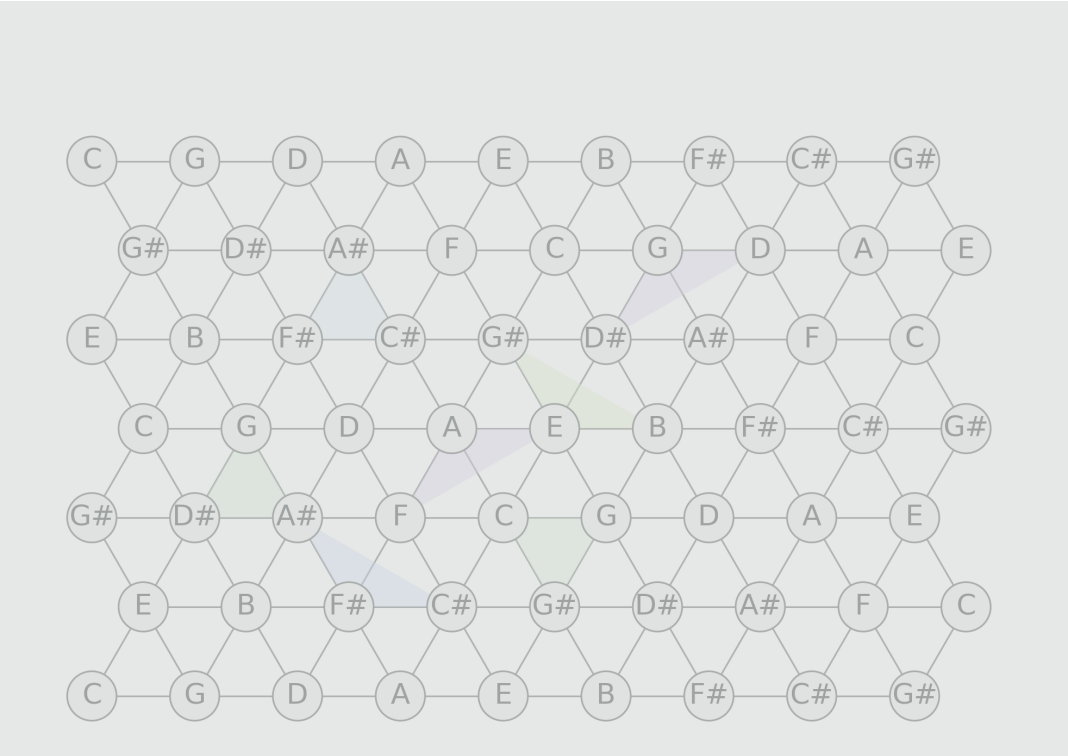


그림 2.4. Tonnetz 격자 구조. 가로 방향은 완전5도(C→G→D→A→E⋯), 대각선 방향은 장3도(C→E→G#⋯)와 단3도(C→A→F#⋯)로 이동한다. 삼각형 하나는 하나의 장3화음(major triad) 또는 단3화음(minor triad)에 대응된다.

(1) Tonnetz 거리. 두 pitch class p_1, p_2 사이의 Tonnetz 거리 $d_T(p_1, p_2)$는 격자 위 최단 경로 길이로 정의된다:

$$d_T(p_1, p_2) = \min\left\{|x_1 - x_2| + |y_1 - y_2| \,\middle|\, (x_i, y_i) \text{ represents } p_i\right\}$$

(2) Voice leading distance (Tymoczko, 2008): 두 pitch class 사이를 이동하기 위해 거쳐야 하는 반음의 개수와 같다.

$$d_V(p_1, p_2) = |p_1 - p_2|$$

(3) DFT distance (Tymoczko, 2008): 각 pitch class를 12차원 벡터로 표현한 뒤, 이산 푸리에 변환(DFT)으로 다른 공간으로 옮겨서 비교한다.

복합 거리(Hybrid distance). 본 연구는 빈도 기반 거리 d_{freq}와 음악적 거리 d_{music} (Tonnetz, Voice leading, DFT 중 하나)을 선형 결합한다:

$$d_{\text{hybrid}}(n_i, n_j) = \alpha \cdot d_{\text{freq}}(n_i, n_j) + (1 - \alpha) \cdot d_{\text{music}}(n_i, n_j)$$

본 연구에서의 사용: 거리 함수의 선택은 발견되는 cycle 구조에 직접적으로 영향을 미친다. 빈도 기반 거리만 사용하면 곡의 통계적 특성만 반영되어 화성적·선율적 의미가 있는 구조를 포착하지 못한다.

주석 — metric 공리와 이론적 정당성. 네 거리 함수 중 Tonnetz와 voice_leading은 metric 공리를 모두 만족하지만, frequency는 시퀀스 전이 빈도 기반 정의로 인해 identity 공리와 삼각 부등식을 위반하며(non-metric), dft는 magnitude-only 설계로 인해 identity 공리를 위반한다(pseudometric — §5.9 transposition-invariance 정리의 의도된 귀결). 이 위반으로 인해 Cohen-Steiner-Edelsbrunner-Harer (2007) stability 정리를 직접 적용할 수 없다. 그러나 Heo-Choi-Jung (2025)의 path-representable distance 프레임워크에 따르면, 완전그래프 K_n 위에 거리 행렬을 edge weight로 부여하는 본 구현은 path-representable + cost-dominated 조건을 자명하게 만족하며, Theorem 3.4에 의해 서로 다른 거리 함수 간 1-dimensional persistence barcode의 **order-preserving injection**이

보장된다. 따라서 frequency/dft가 엄밀한 metric이 아니어도 1-cycle 구조의 탐지와 비교는 이론적으로 정당화된다 ($k \geq 2$ persistence는 Theorem 5.1에 따라 별도 검증 필요).

2.5 활성화 행렬과 중첩행렬

본 연구에서는 곡의 시간축 위에서 cycle 구조가 어떻게 전개되는지를 두 단계의 행렬로 표현한다. 첫 단계는 **활성화 행렬(activation matrix)**, 두 번째 단계는 그것을 가공한 **중첩행렬(overlap matrix, OM)**이다.

정의 2.5 (활성화 행렬). 음악의 시간축 길이를 T, 발견된 cycle의 수를 C라 하자. 활성화 행렬 $A \in \{0,1\}^{T \times C}$는 있는 그대로의 활성 정보를 담는다:

$$A[t,c] = \mathbb{1}[\,\exists\, n \in V(c) \text{ such that } n \text{ is played at time } t\,]$$

여기서 $V(c)$는 cycle c의 note 집합이다. 활성화 행렬은 산발적인 단일 시점 활성화까지 모두 포함하므로 노이즈가 많다.

정의 2.6 (OM). OM $O \in \{0,1\}^{T \times C}$는 활성화 행렬에서 **연속적이고 충분히 긴 활성 구간만 남긴 것**이다.

$$O[t,c] = \mathbb{1}[\,t \in R(c)\,], \qquad R(c) = \bigcup_i [s_i,\ s_i + L_i]$$

여기서 $R(c)$는 cycle c의 "지속 활성 구간(sustained intervals)"의 합집합이며, 각 구간 $[s_i, s_i + L_i]$는 활성화 행렬 $A[\cdot,c]$에서 길이가 임계값 scale_c 이상인 연속 1의 구간이다.

활성화 행렬과 OM의 차이.

- $A[t,c]$: 시점 t에 cycle c의 note가 단 한 번이라도 울리면 1. **순간적 활성을 모두 잡음.**
- $O[t,c]$: cycle c의 활성이 일정 시간 이상 **지속되는 구간**에서만 1. 산발적 노이즈 제거됨.

예를 들어 $\text{scale}_c = 3$일 때 (3 시점 이상 지속된 활성만 인정), 아래와 같다.

```
      :  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15
A[·,c]: 0  1  1  0  1  1  1  1  0  0  1  0  1  1  1
O[·,c]: 0  0  0  0  1  1  1  1  0  0  0  0  1  1  1
```

본 연구에서 OM을 음악 생성의 seed로 사용하는 이유는, 일정 시간 유지되는 cycle만이 곡의 구조적 단위로 의미 있다고 보기 때문이다.

연속값 확장. 본 연구에서는 이진 OM 외에, cycle의 활성 정도를 [0,1] 사이의 실수값으로 표현하는 연속값 버전도 도입하였다:

$$O_{\text{cont}}[t,c] = \frac{\sum_{n \in V(c)} w(n) \cdot \mathbb{1}[\,n \text{ is played at time } t\,]}{\sum_{n \in V(c)} w(n)}$$

여기서 $V(c)$는 cycle c의 vertex 집합, $w(n) = 1/N_{\text{cyc}}(n)$은 note n의 **희귀도 가중치**이며 $N_{\text{cyc}}(n)$은 note n이 등장하는 cycle의 개수이다. 적은 cycle에만 등장하는 note가 활성화되면 더 큰 가중치를 받는다.

음악적 의미: OM은 곡의 **위상적 뼈대(topological skeleton)**를 시각화한 것이다. 시간이 흐름에 따라 어떤 반복 구조가 켜지고 꺼지는지를 나타내며, 이것이 음악 생성의 seed 역할을 한다.

2.6 Jensen-Shannon Divergence — 생성 품질의 핵심 지표

JS divergence는 두 확률 분포가 얼마나 다른지를 측정하는 지표이다. 값의 범위는 $[0, \log 2]$이며, 0이면 두 분포가 동일하다. 값이 낮을수록 두 곡의 음 사용 분포가 유사하다.

본 연구에서 비교하는 두 가지 분포:

1. **Pitch 빈도 분포** — "어떤 음들이 얼마나 자주 쓰였는가" (시간 순서 무시)

2. **Transition 빈도 분포** — "어떤 음 다음에 어떤 음이 오는가" (시간 순서 반영, §5.4~)

두 지표를 함께 사용함으로써 "음을 비슷하게 쓰는가"와 "비슷한 순서로 쓰는가"를 별도로 측정할 수 있다.

2.7 Cycle Subset 선택

본 연구의 DFT baseline ($\alpha = 0.25$)에서는 발견되는 cycle 수가 $C = 14$ 로 소규모이므로, greedy subset selection 없이 전체 cycle을 그대로 사용한다 ($K = C = 14$). Greedy forward selection은 Tonnetz 조건(최대 $C = 47 \sim 52$)에서 cycle subset을 선별하기 위해 개발된 도구이며, 발견 cycle 수가 충분히 작은 DFT baseline에서는 적용하지 않는다.

2.8 Multi-label Binary Cross-Entropy Loss

각 시점에서 동시에 여러 note가 활성화될 수 있으므로, 단일 클래스 예측인 categorical cross-entropy 대신 **multi-label BCE**를 사용한다. 각 note 채널마다 독립적인 binary cross-entropy를 계산하여 "note i 가 활성인가?"를 개별 이진 문제로 학습한다. 모델 입력은 OM의 한 행 $O[t, :] \in \mathbb{R}^C$ 이고, 출력은 N 차원 multi-hot vector이다.

Adaptive threshold: 추론 시 고정 임계값 0.5 대신, 원곡의 평균 ON 비율(약 15%)에 맞춰 sigmoid 출력 상위 15%를 활성으로 채택하는 동적 임계값을 사용한다. 이 임계값은 per-cycle이 아니라 **출력행렬** $P \in \mathbb{R}^{T \times N}$ **전체를 평탄화한 전역 기준**으로 계산한다(§4.2의 OM 임계값 $\tau \in \{0.3, 0.5, 0.7\}$ 실험과는 별개).

2.9 음악 네트워크 구축과 가중치 분리

가중치 행렬의 분리 (본 연구의 핵심 설계):

1. **Intra weights** : 같은 악기(inst) 내에서 연속한 두 화음 간 전이 빈도. 각 악기의 **선율적 흐름**을 포착한다.

$$W_{\text{intra}} = W_{\text{intra}}^{(1)} + W_{\text{intra}}^{(2)}$$

2. **Inter weight** : 시차(lag) ℓ 을 두고 inst 1의 note와 inst 2의 note가 인접하여 출현하는 빈도이다. $\ell \in \{1, 2, 3, 4\}$ 로 변화시키며 다양한 시간 스케일의 **악기 간 상호작용**을 탐색한다. 가까운 시차에 더 큰 비중을 두는 **감쇄 가중치** λ_ℓ 을 사용하여 합산한다:

$$W_{\text{inter}} = \sum_{\ell=1}^{4} \lambda_\ell \cdot W_{\text{inter}}^{(\ell)}, \qquad (\lambda_1, \lambda_2, \lambda_3, \lambda_4) = (0.6,\ 0.3,\ 0.08,\ 0.02)$$

가중치는 "먼 시차의 우연한 동시 등장보다 가까운 시차의 인과적 상호작용이 음악적으로 의미 있다"는 가정을 반영한다.

Timeflow weight (선율 중심 탐색):

$$W_{\text{timeflow}}(r_t) = W_{\text{intra}} + r_t \cdot W_{\text{inter}}$$

$r_t \in [0, 1.5]$ 를 변화시키며 위상 구조의 출현·소멸을 추적한다.

Simul 혼합 모드(기존 complex)는 short 버전에서 공식을 생략하고, §5.9의 결과 요약만 제시한다.

rate parameter r_t 의 의미. r_t 는 timeflow weight에서 intra weights와 inter weight의 비중을 조절한다.

- $r_t = 0$: $W = W_{\text{intra}}$ 만 사용. 각 악기의 선율적 흐름만 반영.
- $r_t = 1$: intra와 inter를 동등하게 결합. 선율과 상호작용을 균형 있게 반영.
- $r_t > 1$: inter의 비중이 intra보다 커짐. 악기 간 상호작용이 지배적인 구조를 탐색.

3. **Simul weight** : 같은 시점에서 inst 1과 2가 동시에 타건하는 note 조합의 빈도. **순간적 화음 구조**를 포착한다.

Complex weight (선율-화음 결합):

$$W_{\text{complex}}(r_c) = W_{\text{timeflow, refined}} + r_c \cdot W_{\text{simul}}$$

$r_c \in [0, 0.5]$로 제한하여 "음악은 시간 예술이므로 화음보다 선율에 더 큰 비중을 둔다"는 음악적 해석을 반영한다.

Figure 7. 두 악기의 모듈 반복 — Inst 1은 연속, Inst 2는 모듈마다 쉼

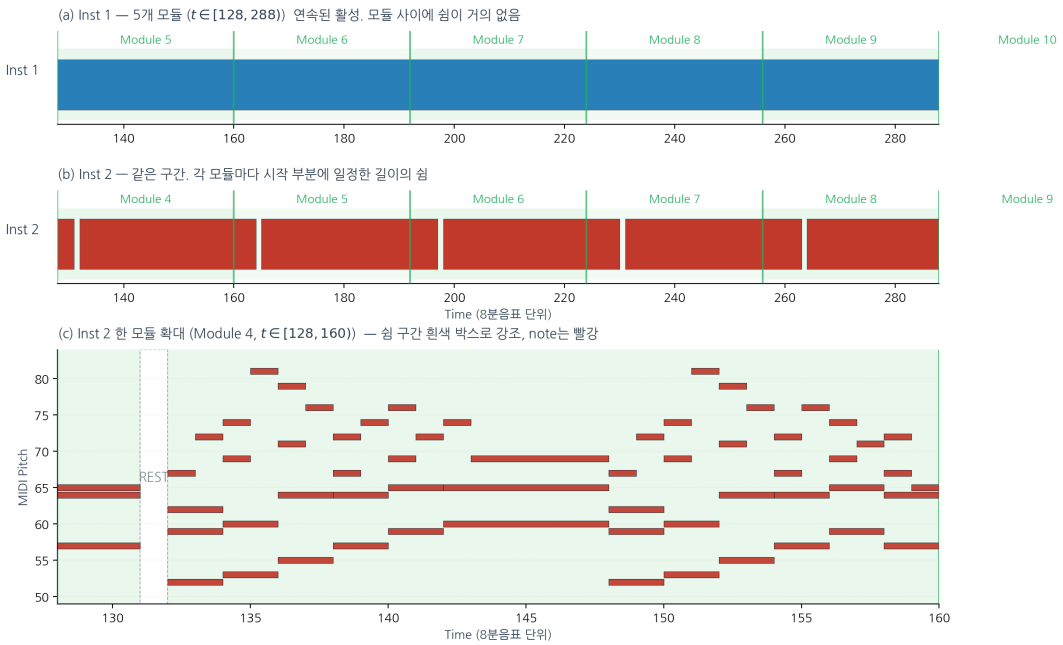


그림 2.9. hibari의 inst 1, 2 배치 구조. inst 1 (위)에선 전체 34마디($T = 1{,}088$ 시점) 중 33마디에서 모듈이 쉬지 않고 연주되며, inst 2 (아래)에선 모듈의 사이마다 1시점의 규칙적인 쉼을 두고 연주된다. 이 비대칭 배치가 가중치 행렬의 intra / inter / simul 분리의 근거가 된다.

2.10 확장 수학적 도구 — 거리 보존 재분배와 화성 제약

본 절은 §5의 확장 실험에서 사용되는 도구를 간략히 소개한다.

Persistence Diagram Wasserstein Distance: 두 barcode의 birth-death 점들을 최적 매칭한 이동 비용. 두 위상 구조의 유사도를 직접 비교하는 데 사용한다.

Consonance score: 시점별 동시 타건 note 쌍의 roughness(불협화도) 평균. 음악이론의 협화도 분류에 기반하여, 생성된 음악의 화성적 질을 평가한다.

Markov chain 시간 재배치: 원본 OM $O \in \{0,1\}^{T \times K}$의 각 행을 state(multi-hot cycle 활성 벡터)로 보고, 시점 $t-1 \to t$ 전이 빈도로 전이확률 $P(s_t \mid s_{t-1})$를 추정한 뒤, 온도 τ로 재샘플링하여 새로운 OM 시퀀스를 생성하는 기법. OM 자체는 학습 대상이 아니라 생성 시 조건부 입력이며, Markov는 그 OM의 시간 전개 패턴을 추정·변형한다.

---

3. 두 가지 음악 생성 알고리즘

표기 정의

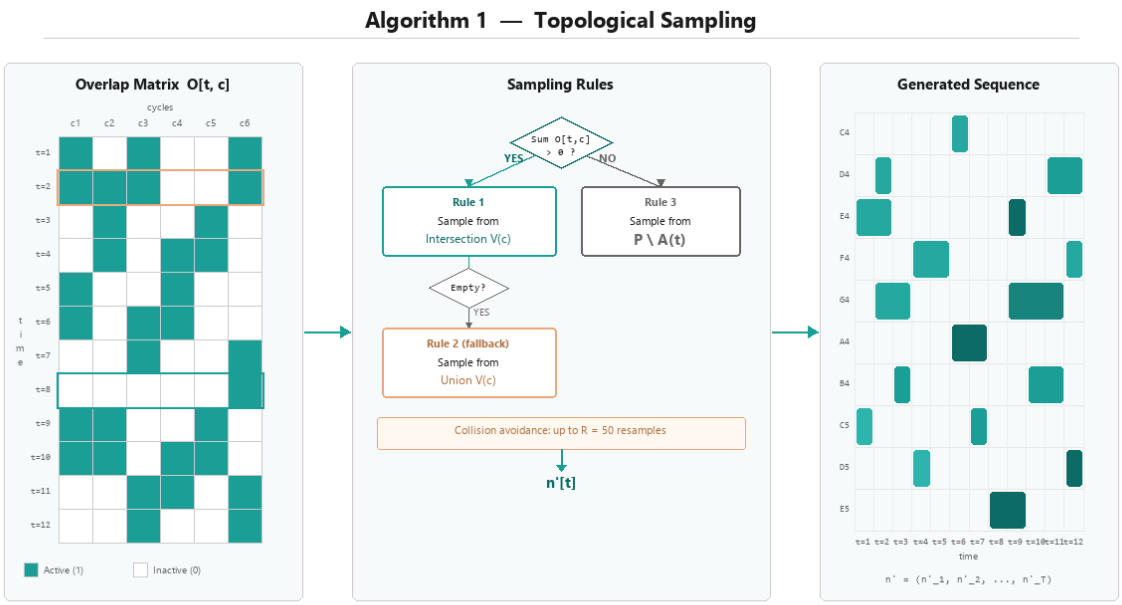
본 장에서 사용할 표기를 다음과 같이 통일한다.

기호	의미	hibari 값
T	시간축 길이 (8분음표 단위)	$1{,}088$
N	고유 note 수 (pitch-duration 쌍)	23

C	발견된 전체 cycle 수	14 (DFT $\alpha = 0.25$) / 19 (DFT $\alpha = 0.5$)
K	분석에 사용한 cycle 수	§4~§4.3: $K = 14$; §5.8.1: $K = 19$ ($\alpha = 0.5$); §6: $K = 14$
O	OM, $\{0,1\}$ 값의 $T \times K$ 행렬	—
L_t	시점 t에서 추출할 note 개수	3 또는 4
$V(c)$	cycle c의 vertex(note label) 집합	원소 수 $4 \sim 6$
R	재샘플링 최대 시도 횟수	50
B	학습 미니배치 크기	32
E	학습 epoch 수	200
H	DL 모델의 hidden dimension	128
bar	음악적 마디 (4/4 기준 8 timestep)	8 timestep
module	hibari 반복 선율 단위 (A-B-A'-C), 32 timestep = 4마디	inst 1에서 33회 반복

3.1 Algorithm 1 — 확률적 샘플링 기반 음악 생성

> **참고:** Algorithm 1의 샘플링 규칙 1, 2는 선행연구(정재훈 외, 2024)에서 설계된 것이며, 본 연구는 이를 계승하여 사용한다.



핵심 아이디어 (3가지 규칙)

규칙 1 — 시점 t에서 활성 cycle이 있는 경우, 즉

$$\sum_{c=1}^{K} O[t, c] > 0$$

일 때, 활성화되어 있는 모든 cycle들의 교집합 I(t)에서 note 하나를 **균등 추출**한다. 만약 교집합이 공집합이면, 활성 cycle들의 합집합 U(t)에서 균등 추출한다.

$$I(t) = \bigcap_{c\,:\,O[t,c]=1} V(c) \qquad\qquad U(t) = \bigcup_{c\,:\,O[t,c]=1} V(c)$$

규칙 2 — 시점 t에서 활성 cycle이 없는 경우, 즉

$$\sum_{c=1}^{K} O[t,c] = 0$$

일 때, 인접 시점 $t-1, t+1$에서 활성화된 cycle들의 vertex의 합집합

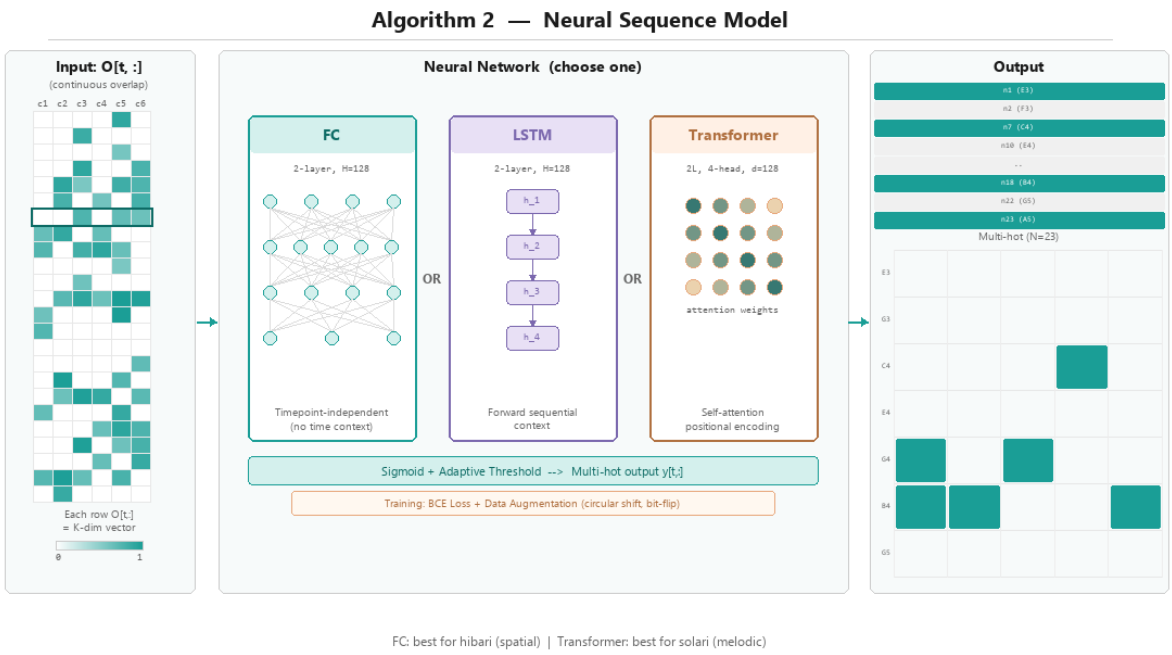
$$A(t) = \bigcup_{c\,:\,O[t-1,c]=1} V(c) \cup \bigcup_{c\,:\,O[t+1,c]=1} V(c)$$

을 계산한 뒤, 전체 note pool P에서 이 합집합을 제외한 영역 $P \setminus A(t)$에서 균등 추출한다.

규칙 3 — 중복 onset 방지. 같은 시점 t에서 동일한 (pitch, duration) 쌍이 두 번 추출되지 않도록 검사하며, 충돌이 발생하면 최대 R회까지 재샘플링한다. R회 모두 실패하면 그 시점의 해당 note 자리는 비워둔다.

3.2 Algorithm 2 — 신경망 기반 시퀀스 음악 생성

> **참고:** Algorithm 2의 전체 구조는 아래 그림에 시각적으로 요약되어 있다. FC / LSTM / Transformer 세 아키텍처 중 하나를 선택하여 사용한다.



알고리즘 개요

Algorithm 2는 OM을 입력, 원곡의 multi-hot note 행렬을 정답 레이블로 두고 매핑

$$f_\theta : \{0,1\}^{T\times C} \longrightarrow \mathbb{R}^{T\times N}$$

을 학습한다 (FC 모델은 시점별 독립이므로 $\{0,1\}^C \to \mathbb{R}^N$). 학습된 모델은 학습 시 보지 못한 cycle subset이나 노이즈가 섞인 OM에 대해서도 원곡과 닮은 note 시퀀스를 출력하도록 기대된다.

DL 모델은 Algorithm 1처럼 "교집합 규칙"으로 위상 구조를 직접 강제하지는 않는다. 대신 Subset Augmentation을 통해 $K \in \{10, 15, 20, 30, 46\}$과 같은 다양한 크기의 subset에 대해서도 같은 원곡 y를 복원하도록 학습한다. 이 과정에서 모델은 "서로 다른 cycle subset이 같은 음악을 유도할 때, 그 공통적인 구조적 특성"을 잠재 표현으로 내부화한다. 따라서 학습 시 구체적으로 보지 못한 subset(예: $K=12$)에 대해서도, 모델이 학습한 잠재 표현이 충분히 일반화되어 있다면 합리적 출력이 가능하다.

> **일반화의 범위.** 이 일반화는 동일 cycle set 위에서 변형된 OM(subset 선택, threshold 변경, 시간 재배치 등)에 한정된다. 새로운 cycle set(거리 함수 변경, 다른 곡)이 입력되면 재학습이 필요하다.

모델 아키텍처 비교

본 연구는 동일한 학습 파이프라인 위에서 세 가지 모델 아키텍처를 비교한다.

모델	입력 형태	시간 정보 처리 방식	파라미터 수
FC	(B, C)	시점 독립	4×10^4
LSTM (2-layer)	(B, T, C)	순방향 hidden state	2×10^5
Transformer (2-layer, 4-head)	(B, T, C)	self-attention	4×10^5

표기 설명: B는 batch size(한 번에 묶어서 학습하는 데이터 개수), T는 시간 길이(timestep 수), C는 cycle 수(OM의 열 수)이다.

FC: 시점을 독립적으로 처리하므로 한 번에 시점 하나씩(C차원 벡터)을 입력받아 (B, C) 형태가 된다. 여기서 B는 T개 시점 중 묶어서 처리하는 수이며 기본값 $B=32$이다.

LSTM/Transformer: 곡 전체 시퀀스를 한 번에 입력받으므로 (B, T, C) 형태가 된다. 단, $T=1{,}088$이 batch size $(=32)$보다 훨씬 크므로 ($\lfloor 32/1{,}088 \rfloor = 0$) 실제로는 한 번에 시퀀스 1개씩 처리된다 ($B=1$). Augmentation으로 생성된 변형본들은 배치 크기를 늘리는 것이 아니라, epoch 내 학습 스텝 수를 늘린다.

학습 손실 함수

§2.8에서 정의한 binary cross-entropy 손실을 사용한다.

추론 단계

학습이 끝난 모델 f_{θ^*}로 새로운 음악을 생성하는 단계를 하나하나 풀어 설명한다.

1단계 — 모델 통과 : logit 생성.

2단계 — sigmoid 변환.

3단계 — adaptive threshold 결정. 가장 단순한 방법은 " $P[t,n] \geq 0.5$이면 켠다"라고 고정 임계값을 쓰는 것이다. 그러나 LSTM이나 Transformer 같은 시퀀스 모델은 sigmoid 출력이 전반적으로 낮게 형성되는 경향이 있어, 0.5를 그대로 쓰면 활성화되는 note가 거의 없어 음악이 텅 비어버린다. 이를 해결하기 위해 본 연구는 원곡의 ON ratio에 맞춰 threshold를 데이터 기반으로 동적 결정한다.

ON ratio란 "원곡의 multi-hot 행렬 $y \in \{0,1\}^{T \times N}$에서 전체 $T \times N$개의 셀 중 값이 1인 셀의 비율"을 뜻한다. 수식으로는

$$\rho_{\text{on}} = \frac{1}{T \cdot N} \sum_{t=1}^{T} \sum_{n=1}^{N} y[t,n]$$

이다. hibari의 경우 $T=1{,}088$, $N=23$이므로 전체 셀 수는 약 $25{,}024$개이고, 그 중 note가 활성인 셀 수를 세어 나누면 약 15% ($\rho_{\text{on}} \approx 0.15$)가 된다. 직관적으로는 "한 시점당 23개 note 중 평균 $3 \sim 4$개가 켜져 있는 정도"라고 이해할 수 있다.

이 ρ_{on}을 목표 활성 비율로 삼아, P의 모든 값 중 상위 15%에 해당하는 경계값 θ를 임계값으로 쓴다. 구현은 per-cycle 기준이 아니라 P 전체($T \times N$)를 평탄화한 전역 top-k 방식이다.

4단계 — note 활성화 판정. 모든 (t, n) 쌍에 대해 $P[t, n] \geq \theta$이면 시점 t에 note n을 활성화한다. 이 note의 (pitch, duration) 정보를 label 매핑에서 복원하여 $(t,\ \text{pitch},\ t + \text{duration})$ 튜플을 결과 리스트 G에 추가한다.

5단계 — onset gap 후처리 (Algorithm 1, 2 공통). 너무 짧은 간격으로 onset이 연속되면 생성된 음악이 지저분하다고 느껴질 수 있다. 그럴 땐 "이전 onset으로부터 일정 시점(`gap_min`) 안에는 새 onset을 허용하지 않는다"는 최소 간격 제약을 적용한다. `gap_min = 0`이면 제약 없음, `gap_min = 3`이면 "3개의 8분음표(= 1.5박) 안에는 새로 타건하지 않음"을 의미한다. 본 연구의 모든 실험에서는 `gap_min = 0`(제약 없음)으로 설정하였다.

이 과정으로 최종적으로 얻은 $G = [(start, pitch, end), \ldots]$를 MusicXML로 직렬화하면 재생 가능한 음악이 된다.

3.3 두 알고리즘의 비교 요약

항목	Algorithm 1 (Sampling)	Algorithm 2 (DL)
학습 필요 여부	불필요	필요 (E epoch)
결정성	확률적 (난수)	결정적 (학습 후)
일반화	같은 곡 내부에서만	보지 못한 cycle subset도 생성
위상 보존 방식	직접 (교집합 규칙으로 강제)	간접 (손실함수를 통해)
생성 시간	약 50 ms	약 100 ms (학습 후)
학습 시간	해당 없음	30 s ~ 3 min

해석. Algorithm 1은 cycle 교집합 규칙을 통해 위상 정보를 직접 강제하므로, 생성된 note의 근거가 "시점 t에 활성화된 cycle들의 교집합"이라는 구조적 규칙으로 투명하게 설명된다 — 설계상 위상 보존이 보장된다. 반면 Algorithm 2는 OM → note 매핑을 학습된 손실함수를 통해 간접적으로 위상을 보존하며 분포의 재현 정확도(§2.6 Jensen-Shannon divergence)가 훨씬 높다: §4.3 FC-cont가 JS $= 0.00035$로 §4.1 Algorithm 1 DFT baseline (0.0213) 대비 약 60배 낮다. 즉 두 알고리즘은 **위상 구조 보존 방식의 투명성**(Algorithm 1)과 **note 분포의 재현 정확도**(Algorithm 2)라는 서로 다른 장점이 있다.

---

4. 실험 설계와 결과

본 장에서는 지금까지 제안한 TDA 기반 음악 생성 파이프라인의 성능을 **정량적으로** 평가한다.

1. **거리 함수 비교** — frequency(기본), Tonnetz, voice leading, DFT 네 종류의 거리 함수에 대해 동일 파이프라인을 적용하고 생성 품질을 비교 (§4.1).
2. **연속값 OM 효과 검증** — 이진 OM 대비 연속값 OM의 효과를 Algorithm 1 및 Algorithm 2 (FC/LSTM/Transformer)에서 검증 (§4.2, §4.3).
3. **DL 모델 비교** — FC / LSTM / Transformer 세 아키텍처를 동일 조건에서 비교하고, continuous overlap을 직접 입력으로 활용하는 효과를 검증 (§4.3).
4. **통계적 유의성** — 각 설정에서 Algorithm 1을 $N = 20$회 독립 반복 실행하여 mean ± std를 보고.

평가 지표

Jensen-Shannon Divergence (주 지표). 생성곡과 원곡의 pitch 빈도 분포 간 JS divergence를 주 지표로 사용한다 (§2.6).

Note Coverage. 원곡에 존재하는 고유 (pitch, duration) 쌍 중, 생성곡에 한 번 이상 등장하는 쌍의 비율. 1.00이면 모든 note가 최소 한 번 이상 사용된 것이다.

거리 함수 구현

두 note 간 확장 — 옥타브와 duration 보정. §2.4의 음악적 거리 함수들은 원래 pitch class만 고려하므로 옥타브와 duration 정보가 손실된다. 본 연구에서 note는 (pitch, duration) 쌍으로 정의되므로, 세 거리 함수 모두에 다음 두 항을 추가한다.

$$d(n_1, n_2) = d_{\text{base}}(p_1, p_2) + w_o \cdot |o_1 - o_2| + w_d \cdot \frac{|d_1 - d_2|}{\max(d_1, d_2)}$$

여기서 d_{base}는 Tonnetz / voice leading / DFT 중 하나, $o_i = \lfloor p_i/12 \rfloor$는 옥타브 번호, d_i는 duration, $w_o = 0.3$, $w_d = 1.0$이다.

각 항의 설계 근거:

- **옥타브 항** $w_o|o_1 - o_2|$: 같은 pitch class라도 옥타브가 다르면 음악적으로 다른 역할을 한다(예: C4와 C5).
- **Duration 항** $w_d|d_1 - d_2|/\max(d_1, d_2)$: 분자를 $\max$로 정규화하여 $[0, 1]$ 범위로 만든다.
- **계수 최적화:** $w_o = 0.3$ (§4.1a)과 $w_d = 1.0$ (§4.1b)은 hibari DFT 조건 N=10 grid search로 최적화되었다.

> **한계 및 후속 과제 — GCD tie 정규화로 인한 w_d 해석의 제약.** 본 파이프라인은 모든 곡에 GCD 기반 pitch-only labeling을 적용하여 duration을 최소 단위로 정규화한다. 이 과정에서 긴 음은 짧은 음의 붙임줄(tie)로 환원되므로 원 악보의 duration 다양성이 축소된다. hibari도 이 정규화를 거치지만 원래 note duration 다양성이 낮아(23개 고유) 실질적 영향이 제한적이다. aqua·solari에서는 note 수를 줄이기 위해 tie 정규화를 전면 적용하여 모든 duration이 GCD 단위(=1)로 수렴하므로 $|d_1 - d_2| = 0$이 되어 duration 항이 비활성화된다. 결과적으로 생성된 음악이 원곡의 리듬·지속 구조를 재현한다고 주장할 수 없으며, w_d grid search는 거리행렬 내 duration 축의 상대적 기여도에 한정된 해석이다. **원 duration을 보존하는 파이프라인 설계는 후속 과제**로 남긴다.

4.1 Experiment 1 — 거리 함수 비교 ($N = 20$)

네 종류의 거리 함수 각각으로 사전 계산한 OM을 로드하여, Algorithm 1을 $N = 20$회 독립 반복 실행하고 JS divergence의 mean ± std를 측정한다.

거리 함수	발견 cycle 수	JS Divergence (mean ± std)	Note Coverage
frequency (baseline)	1	0.0344 ± 0.0023	0.957
Tonnetz	47	0.0493 ± 0.0038	1.000
voice leading	19	0.0566 ± 0.0027	0.989
DFT	17	0.0213 ± 0.0021	1.000

해석 1 — DFT가 가장 우수. DFT 거리 함수는 frequency 대비 JS를 $0.0344 \to 0.0213$으로 낮추어 약 38.2% 낮은 JS를 달성하였다. DFT 거리는 각 note의 pitch class를 $\mathbb{Z}/12\mathbb{Z}$ 위의 12차원 이진 벡터로 표현한 후 이산 푸리에 변환(DFT)을 적용하고, 복소수 계수의 **magnitude(크기)만** 거리 계산에 사용한다. 이때, 이조(transposition)는 DFT 계수의 **위상(phase)**만 바꿀 뿐 magnitude에는 영향을 주지 않는데 이렇게 위상 정보를 버림으로써 이조에 불변인 **화성 구조의 지문**을 추출하게 된다.

특히 $k = 5$ 푸리에 계수가 diatonic scale과 강하게 반응하는 이유는 다음과 같다. 12개 pitch class를 **완전5도 순환(circle of fifths)** 순서 $\{C, G, D, A, E, B, F\sharp, C\sharp, G\sharp, D\sharp, A\sharp, F\}$로 재배열하면, diatonic scale에 속하는 7개 pitch class는 이 순환 상의 **연속된 7개 위치**를 차지한다 (예: C major는 F-C-G-D-A-E-B의 연속 구간). DFT의 $k = 5$ 계수의 magnitude는 정확히 이 "5도 순환 상의 연속성"을 수치로 측정하는 양이며 (Quinn, 2006; Tymoczko, 2008), $\binom{12}{7} = 792$개의 7-note subset 중 diatonic scale 류가 $|F_5|$를 최대화한다 (maximally even property). hibari의 7개 PC 집합 $\{0, 2, 4, 5, 7, 9, 11\}$ (C major / A natural minor) 역시 이 최대화 subset 중 하나이므로, DFT magnitude 공간에서 hibari의 note들은 frequency 거리에서는 포착되지 않던 **음계적 동질성**에 의해 서로 가깝게 군집된다.

해석 2 — 거리 함수가 위상 구조 자체를 바꾼다. "거리 함수의 선택이 곧 어떤 음악적 구조를 '동치'로 간주할 것인가를 정의한다"는 음악이론적 관점과 일치한다.

해석 3 — Note Coverage는 대부분의 설정에서 포화. 원곡의 모든 note 종류가 생성곡에 최소 한 번 등장해야 한다는 기본 요구는 모두 만족된다. 따라서 품질의 주된 차이는 "같은 note pool을 얼마나 자연스러운 비율로 섞는가"에서 발생한다.

4.1a Octave Weight 튜닝 — DFT + N=10 Grid Search

DFT 거리 함수의 옥타브 가중치 w_o를 $\{0.1, 0.3, 0.5, 0.7, 1.0\}$에 대해 hibari Algo1 JS로 N=10 반복 실험하였다. $w_d = 1.0$ (§4.1b 최적값) 고정.

w_o	K (cycle 수)	JS (mean ± std)
0.1	14	0.0380 ± 0.0026
0.3	**19**	0.0163 ± 0.0014
0.5	16	0.0183 ± 0.0020
0.7	16	0.0231 ± 0.0019
1.0	16	0.0204 ± 0.0020

결론: $w_o = 0.3$이 최적이다 (JS = 0.0163). 옥타브 패널티를 줄이면 pitch class 유사성이 거리 행렬을 더 강하게 지배하며, 이는 hibari의 좁은 옥타브 범위(52–81, 최대 2 옥타브)에서 옥타브 구분이 상대적으로 덜 중요하다는 음악적 직관과 일치한다.

4.1b Duration Weight 튜닝 — DFT + N=10 Grid Search

$w_d \in \{0.0, 0.1, 0.3, 0.5, 0.7, 1.0\}$에 대해 hibari + **DFT** 조건에서 N=10 반복 실험을 수행하였다. $w_o = 0.3$ (§4.1a 최적값) 고정.

w_d	K (cycle 수)	JS (mean ± std)
0.0	10	0.0503 ± 0.0042
0.1	25	0.0311 ± 0.0021
0.3	16	0.0221 ± 0.0017
0.5	17	0.0215 ± 0.0024
0.7	17	0.0211 ± 0.0016
1.0 ★	19	0.0156 ± 0.0012

결론: $w_d = 1.0$이 최적이다 (JS = 0.0156). duration 가중치를 최대화할 때 성능이 가장 좋다. DFT는 pitch class 집합의 스펙트럼 구조(indicator vector $\chi_S \in \{0,1\}^{12}$의 이산 Fourier 계수 $|\hat{\chi}_S(k)|$가 나타내는 대칭성 패턴)를 정밀하게 포착하므로 w_d가 높아도 pitch 정보가 충분히 보존되며, 오히려 duration이 거리 행렬에 많이 기여할수록 cycle 수와 생성 품질이 향상된다. 또한 $w_d = 0.0$ (duration 항 완전 제거)일 때 cycle 수가 10으로 급감하고 JS가 0.0503으로 크게 악화되어, duration 정보가 거리 행렬의 질에 유의미하게 기여함을 확인하였다.

4.1c 감쇄 Lag 가중치 실험

§2.9에서 언급한 감쇄 합산 inter weight의 실험적 근거를 제시한다. lag 1 단일 옵션과 lag 1~4 감쇄 합산 옵션 두 설정을 비교하되, 거리 함수는 DFT와 Tonnetz 두 가지를 대조하여 거리 함수의 특성에 따라 효과가 달라짐을 확인한다.

실험 설정:

lag 1 단일 : $W_{\text{inter}} = W_{\text{inter}}^{(1)}$

lag 1~4 감쇄 합산 :

$$W_{\text{inter}} = \sum_{\ell=1}^{4} \lambda_\ell \cdot W_{\text{inter}}^{(\ell)}, \qquad (\lambda_1, \lambda_2, \lambda_3, \lambda_4) = (0.60,\ 0.30,\ 0.08,\ 0.02)$$

고정 조건: hibari, Algorithm 1, N=20

$N = 20$ 반복.

곡	거리 함수	lag 1 단일 (JS mean ± std)	lag 1~4 감쇄 합산 (JS mean ± std)	변화
hibari	DFT	0.0211 ± 0.0021	0.0196 ± 0.0022	-7.1% ★
hibari	Tonnetz	0.0488 ± 0.0040	0.0511 ± 0.0039	$+4.8\%$

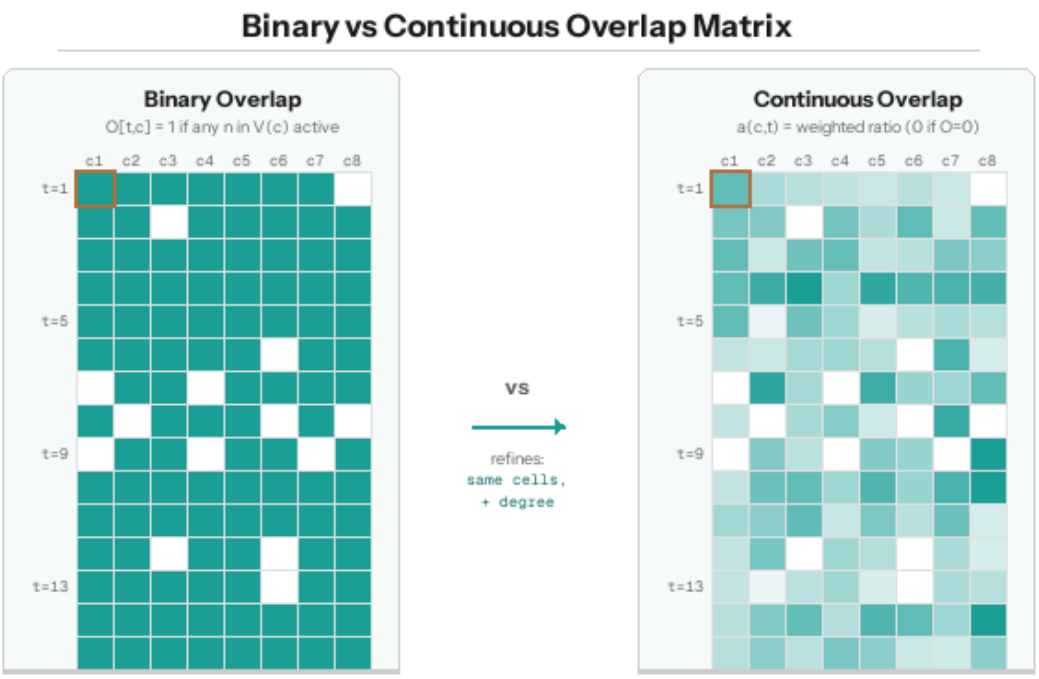
해석 4 — DFT에서 개선, Tonnetz에서는 악화. DFT 거리에서는 감쇄 lag가 JS를 7.1% 개선한다. 반면 Tonnetz에서는 오히려 4.8% 악화한다. 경험적으로 DFT 조건에서만 감쇄 lag 합산이 유효하며, Tonnetz에서는 역효과였다. DFT의 pitch class 스펙트럼 표현이 먼 lag의 시간 상호작용 정보를 흡수하는 데 더 적합하다는 경험적 관찰이며, 이론적 설명은 후속 과제로 남긴다.

> **§4.1c에서 Tonnetz 결과를 병기하는 이유.** DFT가 본 연구의 baseline이지만, §5.6.1 Tonnetz 기반 통합 실험의 수치적 기반을 제공하고, §5.6.3 메타 통찰("감쇄 lag의 효과가 거리 함수에 따라 반대 방향으로 작용")을 실증하기 위해 두 거리 함수를 대조한다.

> **Algorithm 2 미실험 이유.** 감쇄 lag는 OM 생성 이전 단계인 가중치 행렬 W_{inter}에 적용된다. Algorithm 2는 이미 완성된 OM을 입력으로 받으므로, §4.3 DFT baseline 실험의 입력 OM에 감쇄 lag가 묵시적으로 반영되어 있다. 독립 ablation은 별도로 수행하지 않았다.

> **향후 과제 — inter 감쇄 계수 재탐색.** 본 실험에서 $\lambda_1 = 0.60$으로 설정하였는데, intra weight에서 lag=1 계수를 1.0으로 두는 것과의 비대칭이 존재한다. inter lag 감쇄 계수 $(\lambda_1, \lambda_2, \lambda_3, \lambda_4)$는 휴리스틱으로 설정된 것이므로, 향후 $\lambda_1 = 1.0$을 포함한 grid search 또는 uniform/exponential decay 비교 실험이 가능하다.

4.2 Continuous Overlap Matrix 실험



본 절은 §2.5에서 정의한 **연속값 OM** $O_{\text{cont}} \in [0,1]^{T \times K}$가 이진 OM $O \in \{0,1\}^{T \times K}$ 대비 어떤 영향을 주는지를 정량적으로 검증한다. 거리 함수는 모든 설정에서 **DFT**로 고정한다. **본 실험은 Algorithm 1에 대해서만 수행하였다.** Algorithm 2(DL)에서 이진/연속값 입력의 효과는 §4.3에서 세 모델(FC/LSTM/Transformer) 비교 실험의 일부로 다룬다.

실험 설계

cycle별 시점 활성도 $a_{c,t}$는 두 가지 방식으로 계산할 수 있다.

이진 (binary): 단순 OR 연산이다. $V(c)$에 속하는 note가 시점 t에 하나라도 활성이면 $a_{c,t} = 1$, 그렇지 않으면 0이다.

연속값 (continuous): cycle을 구성하는 note들이 얼마나 많은 비율로 활성화되어 있는지를 $[0,1]$ 실수로 표현한다 :

$$a_{c,t} = \Big(\sum_{n \in V(c)} w(n) \cdot \mathbb{1}[n \in A_t] \Big) \Big/ \Big(\sum_{n \in V(c)} w(n) \Big)$$

여기서 A_t는 시점 t에 활성인 note들의 집합, $w(n) = 1/N_{\text{cyc}}(n)$은 note n의 **희귀도 가중치**이며 $N_{\text{cyc}}(n)$은 note n이 등장하는 cycle의 개수이다.

연속값 활성도가 만들어진 후, 최종 OM을 만드는 방식에 따라 다시 두 가지 변형이 가능하다.

- **직접 사용 (direct)**: $O[t,c] = a_{c,t} \in [0,1]$
- **임계값 이진화 (threshold τ)**: $O[t,c] = \mathbb{1}[a_{c,t} \geq \tau]$, $\tau \in \{0.3, 0.5, 0.7\}$

결과

DFT 조건, $w_o = 0.3$, $w_d = 1.0$, $N = 20$ 반복.

설정	Density	JS Divergence (mean ± std)
(A) Binary	0.313	0.0157 ± 0.0018 ★
(B) Continuous direct	0.728	0.0186 ± 0.0015
(C) Continuous → bin $\tau = 0.3$	0.367	0.0360 ± 0.0029
(C) Continuous → bin $\tau = 0.5$	0.199	0.0507 ± 0.0027
(C) Continuous → bin $\tau = 0.7$	0.060	0.0449 ± 0.0024

여기서 "Density"는 **전체 OM (1,088 timestep × K cycle) 기준** 활성 셀의 평균 비율 ($\bar{O}$)이다. §3.2의 **ON ratio** $\rho_{\text{on}} \approx 0.15$와 혼동하지 않도록 주의 — ON ratio는 원곡 multi-hot 행렬 $y \in \{0,1\}^{T \times N}$ 기준이고, Density는 OM $O \in \{0,1\}^{T \times K}$ 기준으로 대상 행렬과 차원이 다르다.

해석

해석 5a — Binary가 최우수. DFT 거리는 스펙트럼 구조를 정밀하게 포착하므로 이진 표현만으로도 cycle 활성 신호가 충분히 구별된다. DFT 이진 OM의 density 0.313은 **선택적 sparsity** — 즉 의미 있는 시점에만 해당 cycle이 활성화되는 중간 상태 — 를 자체적으로 달성한다는 뜻이다. Algorithm 1의 교집합 sampling은 density가 너무 낮으면 활성 cycle이 부족해 note 선택의 다양성이 떨어지고, 너무 높으면 cycle 간 교집합이 비어 fallback(전체 pool 균등 추출)된다.

해석 5b — Continuous direct는 오히려 열세. (B) continuous direct ($\bar{O} = 0.728$)는 이진보다 훨씬 dense하여, Algorithm 1의 교집합 sampling이 과도하게 자주 호출되어 Binary 대비 18.5% 악화한다.

해석 5c — 임계값 이진화는 모두 열세. $\tau = 0.3 \sim 0.7$의 임계값 이진화는 모두 크게 열세다 (JS $0.036 \sim 0.051$). DFT binary OM이 이미 cycle 활성의 핵심 구간만을 선별하므로 추가적인 임계값 필터링은 과도한 sparsity를 만들어 성능을 저하시킨다.

4.3 Experiment 3 — Algorithm 2 DL 모델 비교

세 모델을 **DFT 기반 OM** 입력 ($w_o = 0.3$, $w_d = 1.0$)에서 비교한다. 각 모델을 이진 OM $O \in \{0,1\}^{T \times K}$과 연속값 OM $O_{\text{cont}} \in [0,1]^{T \times K}$의 두 입력에서 모두 학습하였다. $N = 10$ 반복.

모델 아키텍처

FC (Fully Connected, 2-layer, hidden dim $= 128$, dropout $= 0.3$): 각 시점 t의 cycle 활성 벡터 $O[t,:] \in \{0,1\}^K$ 또는 $O_{\text{cont}}[t,:] \in [0,1]^K$를 입력으로 받아 동시점의 note label 분포를 출력. **시점 간 독립 매핑**이므로 시간 문맥 없음.

LSTM (2-layer, hidden dim $= 128$): cycle 활성 벡터 시퀀스 $\{O[t,:]\}_{t=1}^{T}$를 순차 입력. 재귀 구조 $h_t = f(x_t, h_{t-1})$로 과거 문맥을 hidden state에 누적.

Transformer (2-layer, 4-head self-attention, $d_{\text{model}} = 128$): positional embedding으로 시간 위치를 인코딩하고 self-attention으로 **전 시점 동시 참조**가 가능.

세 모델 모두 동일한 학습 조건에서 multi-label BCE loss (§2.8)로 학습한다. 각 시점에서 예측된 note 확률로부터 샘플링하여 생성곡을 얻는다. **validation loss**는 학습 시점 validation set에서 측정한 BCE loss의 평균이다.

모델	입력	Pitch JS (mean ± std)	validation loss (mean)
FC	binary	0.00217 ± 0.00056	0.339
FC	**continuous**	0.00035 ± 0.00015 ★	0.023
LSTM	binary	0.233 ± 0.029	0.408
LSTM	continuous	0.170 ± 0.027	0.395
Transformer	binary	0.00251 ± 0.00057	0.836
Transformer	continuous	0.00082 ± 0.00026	0.152

해석

해석 6 — FC + continuous 입력이 최우수. FC가 연속값 입력에서 JS 0.00035로 가장 낮은 값을 달성하였다. 동일한 FC의 이진 입력 (0.00217) 대비 83.9% 개선된 수치이며, Welch's t-test $p = 1.50 \times 10^{-6}$로 통계적으로 유의하다. Transformer continuous (0.00082)와의 비교에서도 FC continuous가 유의하게 우수하다 (Welch $p = 1.66 \times 10^{-4}$). FC의 cell-wise 표현력이 DFT continuous OM의 cycle 활성 강도 차이를 세밀하게 반영한다.

해석 7 — LSTM의 심각한 열화. LSTM은 이진 입력 0.233, 연속값 입력 0.170으로 다른 두 모델과 비교할 수 없는 수준으로 열화하였다. LSTM의 **재귀 구조(recurrent structure)** 란 시점 t의 hidden state h_t가 직전 시점 h_{t-1}로부터 $h_t = f(x_t, h_{t-1})$로 갱신되는 순차적 정보 전파 메커니즘을 의미하며, 이 구조는 시점이 연속적으로 이어지는 부드러운 시계열(예: 자연어, 음향 파형)에 적합하다. 그러나 OM은 본질적으로 시점별 이산 on/off 활성 패턴이며, 특히 hibari의 cycle 활성 위치는 모듈 기반 phase shifting 구조(§4.5.4)를 따라 모듈 단위로 평행 이동하므로 직전 시점의 hidden state가 현재 시점 예측에 기여하는 바가 작다.

해석 8 — 연속값 입력의 보편적 이점. 세 모델 모두 연속값 입력이 이진 입력보다 유의하게 우수하다 (Welch $p < 10^{-4}$ 전부). 그러나 그 이점의 크기는 모델별로 다르다: FC가 -83.9%로 가장 큰 개선을, Transformer는 -67.4%, LSTM은 -27.3%의 개선을 보인다.

해석 9 — Validation loss와 JS의 동시 감소. FC-continuous는 validation loss (0.023)가 FC-binary (0.339)의 약 7% 수준이며 JS도 동시에 크게 감소한다. 연속값 입력이 모델의 학습 signal을 더 부드럽게 만들어, 과적합 없이 일반화된 분포를 학습하게 한다.

통합 비교 (§4.1 ~ §4.3)

실험	설정	JS divergence	출처
§4.1 Algo 1	frequency baseline	0.0344	§4.1
§4.1 Algo 1	DFT (최적)	0.0213	§4.1
§4.2 Algo 1	DFT binary (최적 파라미터)	0.0157 ± 0.0018 ★	§4.2

§4.3 Algo 2 FC	DFT continuous	0.00035 ± 0.00015 ★	§4.3

§4.3 FC (DFT continuous)는 DFT 기반 실험 내에서 관측된 최저 JS divergence이다. 이론적 최댓값 $\log 2 \approx 0.693$의 약 0.05%에 해당한다.

4.4 종합 논의

(1) 음악이론적 거리 함수의 중요성. §4.1. Experiment 1 결과는 "DFT처럼 음악이론적 구조를 반영한 거리가 더 좋은 위상적 표현을 만들 수 있다"는 가설을 지지한다.

(2) Algorithm 1의 이진 OM, Algorithm 2의 연속값 OM. 거리 함수 baseline으로 DFT를 채택했을 때 Algorithm 1에서는 이진 OM이 연속값 OM보다 우수한 반면 (§4.2), Algorithm 2의 FC에서는 연속값 OM이 큰 폭으로 우수하다 (§4.3, -83.9%). 규칙 기반 Algorithm 1은 sparse한 이진 활성 패턴을 직접 샘플링에 사용하므로 이진 OM이 적합하지만, DL 기반 Algorithm 2의 FC는 continuous activation의 강도 정보를 학습 signal로 활용하여 cell-wise 표현력이 극대화된다.

(3) FC + continuous 입력의 결정적 우위. DL 모델 비교 (§4.3)에서 FC-cont가 JS 0.00035로 Transformer-cont (0.00082) 대비 유의하게 우수하다 (Welch $p = 1.66 \times 10^{-4}$).

4.5 곡 고유 구조 분석 — hibari 의 수학적 불변량

본 절은 hibari 가 가지는 수학적 고유 성질을 분석하고, 이 성질들이 본 연구의 실험 결과와 어떻게 연결되는지를 서술한다. 비교 대상으로 사카모토의 다른 곡인 solari 와 aqua 를 함께 분석한다.

4.5.1 Deep Scale Property — hibari 의 pitch class 집합이 갖는 대수적 고유성

hibari 가 사용하는 7개 pitch class 는 $\{0, 2, 4, 5, 7, 9, 11\} \subset \mathbb{Z}/12\mathbb{Z}$이다. 이 7개 pitch class 집합 전체의 **interval vector**는 $[2, 5, 4, 3, 6, 1]$이다. 여기서 k번째 성분은 "집합 안에서 interval class k에 해당하는 쌍의 수"이다. 7 반음 이상은 옥타브 대칭에 의해 $12 - k$와 동치이므로 interval class는 1~6까지만 존재한다.

이 벡터의 6개 성분은 **모두 다른 수**($\{1, 2, 3, 4, 5, 6\}$의 순열)로 이것을 **deep scale property** 라 한다 (Gamer & Wilson, 2003). 이 성질을 갖는 7-note subset 은 $\binom{12}{7} = 792$개 중 **diatonic scale 류 뿐**이다.

또한 7개 PC 사이의 간격 패턴은 $[2, 2, 1, 2, 2, 2, 1]$로, 오직 $\{1, 2\}$ 두 종류의 간격만으로 구성된다. 이것은 **maximal evenness** — 12개 칸 위에 7개 점을 가능한 한 균등하게 배치한 상태 — 를 의미한다 (Clough & Douthett, 1991). deep scale 과 maximal evenness 는 모두 diatonic scale 의 고유 성질이다.

solari 와 aqua 는 12개 PC 모두를 사용하므로 이 성질이 적용되지 않는다.

4.5.2 근균등 Pitch 분포 — Pitch Entropy

곡	사용 pitch 수	정규화 pitch entropy	해석
hibari	17	0.974	거의 완전 균등
solari	34	0.905	덜 균등
aqua	51	0.891	가장 치우침

pitch entropy는 곡 안에서 사용된 모든 pitch의 빈도 분포에 대한 **Shannon entropy**를 계산하고, 이론적 최댓값으로 나눠 정규화한 것이다. hibari 의 0.974 는 **"모든 pitch 를 거의 같은 빈도로 사용"**한다는 뜻이며, 이것은 §4.3 의 "FC 모델 우위"를 뒷받침한다.

4.5.3 Tonnetz 구별력과 Pitch Class 수의 관계

hibari 의 7개 PC 는 Tonnetz 위에서 **하나의 연결 성분**을 이루며, 평균 degree 가 $3.71/6 = 62\%$이다. Tonnetz 이웃 관계는 ± 3 (단3도), ± 4 (장3도), ± 5 (완전5도) 의 세 가지 방향이며, 각 방향이 양쪽으로 작용하므로 최대 6개의 이웃이

가능하다.

예를 들어 C(0) 의 Tonnetz 이웃은 $\{E, F, G, A\}$의 **4개** 이다:

관계	$+$ 방향	$-$ 방향	hibari 에 있는가
단3도 (± 3)	D#(3)	A(9)	A 있음
장3도 (± 4)	E(4)	G#(8)	E 있음
완전5도 (± 5)	G(7)	F(5)	G, F 있음

Tonnetz 그래프의 지름(diameter).

12개 PC를 전부 사용하는 곡 (solari, aqua) 에서는 어떤 두 PC 든 Tonnetz 거리가 ≤ 2이다. 이는 **"가까운 음"과 "먼 음"을 구별할 여지가 거의 없다**는 뜻이다. 반면 hibari에서는 Tonnetz 거리가 $1 \sim 4$ 범위로 분포하여, 가까운 쌍 (예: C-G, 거리 1) 과 먼 쌍 (예: F-B, 거리 3 이상) 이 명확히 구별된다.

단, 이 Tonnetz 구별력은 **음악이론적 거리 함수가 빈도 기반(frequency)보다 유리한 구조적 근거**일 뿐, hibari의 최적 거리 함수가 Tonnetz임을 의미하지는 않는다. §4.1 실험에서는 **DFT가 Tonnetz 대비 -56.8%로 유의하게 우수**하였다. Quinn (2007)에 따르면, diatonic 7음 집합의 indicator vector는 가능한 모든 7음 집합 중 $k=5$ 계수 $|\hat{\chi}(5)|$가 최대가 된다 — diatonic이 완전5도 chain ($F \to C \to G \to D \to A \to E \to B$)으로 생성되기 때문이다. DFT의 $k=5$ 성분은 이 완전5도 대칭을 직접 측정하므로, hibari의 diatonic 구조를 Tonnetz보다 훨씬 정밀하게 수치화한다. Tonnetz는 차선의 후보일 뿐이다. 이 예측은 §5.1의 solari 실험에서 "12-PC에서는 Tonnetz 구별력이 소실되어 frequency와 Tonnetz가 동등해진다"는 형태로 반대 방향으로도 검증된다.

4.5.4 Phase Shifting — inst 1 과 inst 2 의 서로소 주기 구조

그림 2.9 에서 관찰한 패턴을 더 정밀하게 분석하면, 마디 내 쉼의 상대위치가 모듈마다 정확히 1칸씩 이동한다는 것이 발견된다. inst 2 의 모듈별 쉼 위치를 조사하면 **대각선 패턴**이 나타난다. :

```
module  1: rest at position  0
module  2: rest at position  1
module  3: rest at position  2
module  4: rest at position  3
  ...
module  k: rest at position  k-1
```

32-timestep의 마디 안에서 쉼의 위치가 매 모듈마다 정확히 1칸씩 오른쪽으로 밀린다. 32개 모듈을 거치면 쉼이 마디 내 모든 상대위치 ($0, 1, 2, \ldots, 31$) 를 정확히 한 번씩 방문한다.

이 구조는 미니멀리즘 작곡가 Steve Reich 가 Piano Phase (1967) 에서 사용한 **phase shifting** 기법과 수학적으로 동일하다. 같은 패턴을 연주하는 두 악기 중 하나가 아주 조금 느리게 진행하여, 같은 패턴이 점점 어긋나다가 한참 뒤에야 원래대로 정렬되는 것이다.

hibari 에서 이 phase shifting 은 다음과 같이 수치화된다.

	inst 1	inst 2
모듈 주기	$M=32$ (쉼 없음)	$M+1=33$ (32 음 + 1 쉼)
반복 횟수	33 회	32 회
총 길이	$33 \times 32 = 1{,}056$	$32 \times 33 = 1{,}056$

두 주기가 32 와 33 으로 **서로소**이다. 이 관계 때문에 **두 악기의 위상(phase)이 곡 전체에서 모듈 단위로 한 번도 동기화되지 않는다.** 이때, 모듈 자체가 A-B-A'-C 구조로 구성되어 국소적으로 같은 위상이 반복되는 부분이 최소 2번 있기는 하다.

이 구조는 수학적으로 **Euclidean rhythm** (Bjorklund, 2003; Toussaint, 2005) 과도 연결된다. Euclidean rhythm 은 "n 비트 중 k 개를 가능한 한 균등하게 배치" 하는 알고리즘으로, 아프리카 전통 음악과 전자 음악에서 널리 사용된다. hibari 의 경우 "33 칸 중 1 칸을 비운다" 를 32번 반복하면서 매번 1칸씩 이동하는 것이 Euclidean rhythm 의 가장 단순한 형태이다.

음악적 효과. 이 서로소 구조는 §4.5.2 에서 관찰한 근균등 pitch entropy (0.974) 와 일관된 설계 원리이다 — **pitch 선택도 균등하고, 쉼 배치도 균등하다**. 두 악기가 서로소 주기로 배치되어 있다는 사실은, 단순한 겹치기가 아니라 **수론적으로 최적인 위상 분리**가 달성되어 있음을 시사한다.

---

5. 확장 실험

본 연구는 원곡 재현(§3–§4)을 넘어 여러 방향의 확장 실험을 수행하였다. hibari의 **모듈 단위 생성** 구현 및 정량 평가는 §6에서 별도로 다룬다.

5.1 다른 곡으로의 일반화 — solari 실험 결과

Algorithm 1 — 거리 함수 비교

거리 함수	cycle 수	density	JS (mean ± std)	JS min
frequency	22	0.070	0.063 ± 0.005	0.056
Tonnetz	39	0.037	0.063 ± 0.003	0.059
voice leading	25	0.043	0.078 ± 0.004	0.073
DFT	15	0.071	0.0824 ± 0.0029	0.0773

hibari에서는 DFT가 최우수(0.0213)였지만, solari에서는 frequency/Tonnetz가 동률에 가깝고 DFT는 오히려 악화된다. 12-PC 구조에서는 Tonnetz 지름 한계와 함께 DFT의 분해능도 제한된다($K = 15$). 분해능 제한의 원인: 12-PC 전체를 사용하면 indicator vector가 $(1, 1, \ldots, 1)$에 가까워져 비영 Fourier 계수 $|\hat{x}(k)|$가 0에 수렴하고, PC 집합 간 DFT 거리 차이가 소실되어 구조 구별력이 떨어진다.

Algorithm 2 — DL 모델 비교

설정	FC	LSTM	Transformer
이진 OM - JS	0.106	0.168	0.032
연속값 OM - JS	0.042	0.171	0.016

핵심 발견: solari에서는 Transformer가 최적. hibari와 반대 패턴이다. hibari에서 FC 최적 / Transformer 열등이었던 것이, solari에서는 Transformer가 FC의 2.6배 (continuous 기준) 우위이다.

연속값 OM의 효과 solari에서도 연속값 OM은 이진 OM 대비 개선을 보였다. Transformer 기준 binary JS 0.032 → continuous JS 0.016 (50% 감소). 이 개선폭은 hibari의 FC-cont (§4.3, -83.9%)와 같은 방향이나, 크기는 더 작다 (solari Transformer는 -50%). Algorithm 2에 대한 continuous overlap의 효과가 곡의 특성에 독립적임을 시사한다.

곡	PC 수	정규화 entropy	최적 거리	최적 모델	해석
hibari	7 (diatonic)	0.974	DFT	FC	공간적 배치, 시간 무관
solari	12 (chromatic)	0.905	Tonnetz (frequency)	Transformer	선율적 진행, 시간 의존

5.2 클래식 대조군 — Bach Fugue 및 Ravel Pavane

곡 기본 정보

곡	T (8분음표)	N (고유 note) †	화음 수
hibari (참고)	1088	23	17
solari (참고)	224	34	49

Ravel Pavane	548	**49**	230
Bach Fugue	870	**61**	253

† 8분음표 기준 GCD = 1 tie 정규화(같은 pitch를 공유하는 note마다 duration의 최대공약수를 구하여 붙임음(tie)을 통해 연속 타건되었다고 해석) 적용 후 고유 (pitch, dur) 쌍 수.

거리 함수별 Algo1 JS

곡	frequency	tonnetz	voice leading	DFT	최적
hibari	0.0344	0.0493	0.0566	0.0213	**DFT**
solari	0.0634	0.0632	0.0775	0.0824	Tonnetz (frequency)
Ravel Pavane	0.0337	0.0387	0.0798	0.0494	**frequency**
Bach Fugue	0.0902	0.0408	0.1242	0.0951	**Tonnetz**

해석

Ravel Pavane: frequency 최적, 가설 불확인. N=49로 note 다양성이 높은 Ravel에서 빈도 역수 가중치가 가장 효과적이다. Tonnetz는 오히려 JS가 14.8% 악화된다. note 다양성(N=49)이 클수록 빈도 기반 분리자(frequency)가 강점을 갖는다는 가설이 지지된다.

Bach Fugue: Tonnetz 최적, voice leading 최악. Bach Fugue(BWV 847 등)는 **푸가(fugue)** 형식 — 여러 성부가 같은 주제를 모방 진행(imitation)하는 **대위법(counterpoint)**의 대표 사례이다. 수직 방향 선율 진행보다 화성적 Tonnetz 공간 이동이 지배적이므로, 역설적으로 대위법 작품에서도 Tonnetz 거리가 최적으로 작동한다. Tonnetz 거리의 최적성은 post-bugfix 실험(Task 39) 이전의 수치이며, DFT 열 값(0.0951)은 post-bugfix 검증되었다 — Tonnetz 열(0.0408) bugfix 전후 비교는 B 세션 확인 필요.

거리 함수 패턴 종합:

곡	PC 수	최적 거리	해석
hibari	7 (diatonic)	**DFT**	7음계 스펙트럼 구조 ($k=5$ 성분) 포착
aqua	12 (chromatic)	Tonnetz	화성적 공간 배치
Bach Fugue	12 (chromatic)	Tonnetz	화성적 공간 배치
Ravel Pavane	12	frequency	note 다양성 지배
solari	12	Tonnetz (frequency)	12-PC 구별력 한계, Tonnetz frequency

현재 데이터에서 **hibari만 DFT 최적**이며, 나머지 곡(solari/aqua/Bach/Ravel)은 Tonnetz 또는 frequency가 최적이다. 즉 "거리 함수의 절대 최적"이 있는 것이 아니라 곡의 구조와 목적에 따라 최적이 달라진다. 테스트한 5곡 중 voice leading이 최적인 곡은 없다.

> **참고:** Bach Fugue (N=61) 및 Ravel Pavane (N=49)에 대해 Algorithm 2 (DL) 실험은 수행하지 않았다. 이는 본 절의 목적이 거리 함수 선택 효과 검증에 있었기 때문이며 해당 클래식 대조군 두 곡에 대한 Algorithm 2 적용은 후속 연구 과제로 남긴다.

5.3 위상 구조 보존 음악 변주 — 개요

§5.3–§5.6은 "위상 구조를 보존하면서 원곡과 다른 음악을 만드는 변주 실험"이다. 본 장의 기본 축은 **Tonnetz 기반**으로 두며, 이는 scale 제약과 화성적 이웃 구조가 Tonnetz와 가장 잘 맞기 때문이다. 이 가정의 타당성은 §5.6.2에서 DFT 전환 실험으로 직접 검증한다.

지금까지의 모든 실험은 원곡의 pitch 분포를 최대한 재현하는 것을 목표로 했다. 이제 방향을 전환하여, **위상 구조를 보존하면서 원곡과 다른 음악**을 생성하는 문제를 다룬다. 세 가지 접근을 조합한다: (1) OM 시간 재배치 (§5.4), (2) 화성 제약 기반 note 교체 (§5.5), (3) 두 방법의 결합 (§5.6).

평가 지표 정의.

DTW: Dynamic Time Warping (DTW)은 두 pitch 시퀀스 사이의 거리를 측정한다. 본 연구에서 pitch 시퀀스는 **각 note의 onset 시점을 시간순으로 정렬하여 note당 pitch 값 하나씩을 뽑은 리스트**이다. 즉, 길게 지속되는 음도 짧은 음도 시퀀스에 동일하게 한 번만 포함된다. 두 시퀀스 $x=(x_1,\ldots,x_T)$와 $y=(y_1,\ldots,y_S)$에 대해 DTW는 두 시퀀스의 모든 정렬 경로 중 최소 비용을 갖는 것을 선택한다:

$$\mathrm{DTW}(x,y)=\min_{\mathrm{warpingpath}}\sum_{(i,j)\in\mathrm{path}}|x_i-y_j|$$

여기서 **warping path**는 (1) $(1,1)$에서 (T,S)까지 x, y 각 성분이 단조 증가하며 (역방향 불가), (2) 각 단계에서 x, y 각 성분이 최대 1 증가하는 정렬 경로이다. DTW는 시퀀스 길이가 달라도 비교 가능하며, 일반 유클리드 거리와 달리 시간 축의 국소적 신축을 허용하여 선율의 전반적인 윤곽을 비교한다. 원곡과 생성곡의 pitch 진행 패턴이 얼마나 다른지를 측정하는 선율 차이 지표로 사용된다. 값이 클수록 두 곡의 선율이 더 많이 다르다.

§5.3–§5.6의 모든 실험에서 Algorithm 1 및 Algorithm 2는 **원곡 hibari의 OM**을 그대로 사용한다. OM의 구조(몇 개의 note로 구성된 어떤 cycle들이 시계열에서 어떤 양상으로 중첩된 채 활성화되는지)는 원곡과 동일하며, 변주는 언제 연주하느냐(§5.4) 또는 어떤 note를 사용하느냐(§5.5)에서만 발생한다.

5.4 OM 시간 재배치

OM의 **행(시점)**을 재배치하여 같은 cycle 구조를 다른 시간 순서로 전개한다.

3가지 재배치 전략

1. **segment_shuffle**: 동일한 활성화 패턴이 연속되는 구간을 식별하고, 구간 단위로 순서를 셔플. 구간 내부 순서는 유지. 패턴이 바뀌는 시점을 경계로 삼으므로 구간 길이가 가변적이다. hibari DFT(gap=0) 기준 실측에서는 $T=1088$에서 segment 수가 1012개(평균 길이 1.08)로, 시작/끝 일부를 제외하면 사실상 1-step 구간이 대부분이다.
2. **block_permute** (block size 32/64): 고정 크기 블록을 무작위 순열로 재배치.
3. **markov_resample** ($\tau=1.0$): 원본 OM의 전이확률로부터 Markov chain을 추정하고, 온도 τ로 새 시퀀스를 생성 (§2.10).

세 모델의 시간 재배치 반응. FC, LSTM, Transformer는 시간 재배치에 대해 구조적으로 다른 반응을 보인다.

FC: 각 시점 t의 cycle 활성 벡터를 독립 처리하므로, OM 행 순서가 바뀌어도 **pitch 분포는 거의 불변**이다. baseline/segment/block32의 pitch JS가 모두 0.000373 ± 0.000281로 동일했고, markov만 0.001030 ± 0.000087로 미세하게 상승했다. 반면 DTW는 baseline 대비 segment $+47.8\%$, block32 $+30.3\%$, markov $+34.1\%$로 크게 증가해, FC가 "같은 pitch 분포를 유지하면서 선율 시간 순서만 바꾸는" 변주에 구조적으로 적합함을 확인했다.

LSTM: 실측 기준으로 retrain X의 세 전략 DTW 변화가 모두 $\leq 0.5\%$였다 (segment $+0.11\%$, block $+0.12\%$, markov $+0.36\%$). retrain O에서도 segment는 -1.09%로 제한적이다.

Transformer: 명시적 PE로 시간 위치를 학습하므로, **PE 유무**가 재배치 효과의 핵심 변수다.

전략 (FC, DFT, N=5)	pitch JS (mean ± std)	DTW (mean ± std)	DTW 변화 (vs baseline)
baseline	0.000373 ± 0.000281	1.26735 ± 0.01837	—
segment_shuffle	0.000373 ± 0.000281	1.87275 ± 0.01846	$+47.8\%$
block_permute(32)	0.000373 ± 0.000281	1.65110 ± 0.02214	$+30.3\%$
markov ($\tau=1.0$)	0.001030 ± 0.000087	1.69975 ± 0.01166	$+34.1\%$

LSTM은 시간 재배치 반응이 여전히 제한적이다. 이하 실험은 Transformer의 PE 유무 비교를 중심으로 보고, DFT 전환 수치는 §5.6.2에서 통합 비교한다. FC는 분포-보존형 변주 특성으로 §5.6.3에서 메타적으로 다시 해석한다.

평가 지표 보충. transition JS는 두 곡의 note-to-note 전이 분포 간 JS divergence이다.

Transformer 결과 (PE 제거 + 재학습 실험)

Transformer에서 positional embedding(PE)을 제거하고, 재배치된 OM으로 재학습:

설정	pitch JS	transition JS	DTW	DTW 변화
noPE_baseline	0.011	0.128	1.85	—
noPE_segment_shuffle (retrain X)	0.011	0.149	1.87	$+1.0\%$
noPE_markov (retrain X)	0.010	0.138	1.87	$+0.9\%$
noPE+retrain segment_shuffle ★	0.173	0.399	2.22	$+21.7\%$
noPE+retrain markov ($\tau = 1.0$)	0.185	0.443	2.16	$+18.0\%$

딜레마: PE 제거 + 재학습에서 DTW가 $+21.7\%$까지 증가하여 선율이 확실히 바뀌지만, 동시에 pitch JS가 $0.007 \to 0.173$으로 **분포가 붕괴**된다.

약한 재배치 → pitch 보존, 선율 변화 없음

강한 재배치 → 선율 변화, pitch 분포도 붕괴

이 딜레마는 시간 재배치 단독으로는 "pitch 유지 + 선율 변화"를 동시에 달성하기 어려움을 의미한다.

5.5 화성 제약 조건

위상 구조를 보존하면서 note를 교체할 때, 제약 없이 선택하면 결과가 **음악적으로 불협화**할 수 있다. 본 절은 화성(harmony) 제약을 추가하여 note 교체의 음악적 품질을 개선한다.

3가지 화성 제약 방법

1. **scale 제약**: 새 note의 pitch class를 특정 스케일 (major, minor, pentatonic)에 한정. 허용 pool 크기가 줄어들지만 음악적 일관성이 보장된다.
2. **consonance 목적함수**: 재분배 비용에 평균 dissonance (§2.10)를 penalty로 추가:

$$\text{cost} = \alpha_{\text{note}} \cdot \text{dist_err} + \beta_{\text{diss}} \cdot \text{diss}$$

실험 기본값은 $\alpha_{\text{note}} = 0.5$, $\beta_{\text{diss}} = 0.3$이다 (`note_reassign.py`, `run_note_reassign_unified.py`).

여기서 dist_err는 **원곡 note 쌍의 위상 거리와 재분배 note 쌍의 위상 거리 사이의 평균 절대 오차**로,

$$\text{dist_err}(S_{\text{orig}}, S_{\text{new}}) = \frac{1}{\binom{n}{2}} \sum_{i<j} \left| d(n_i^{\text{orig}}, n_j^{\text{orig}}) - d(n_i^{\text{new}}, n_j^{\text{new}}) \right|$$

로 정의된다. 값이 0이면 재분배가 모든 쌍 거리를 완전히 보존한 것이고, 클수록 위상 구조가 왜곡된 것이다. 표 1의 **"note 오차"** 컬럼은 이 dist_err 수치이다. 단위는 사용한 거리 함수에 의존한다 — Tonnetz는 hop 수(정수), voice_leading은 반음 수, DFT는 Fourier 계수 L^2 norm으로 각기 다르므로, **같은 거리 함수 내에서만 dist_err 값을 비교할 수 있다.** 본 §5.5 표는 Tonnetz note metric 기준이다.

3. **interval structure 보존**: 원곡의 interval class vector와 새 note 집합의 ICV 차이를 penalty로 추가. **ICV 차이**는 두 집합의 ICV 벡터 사이의 L^1 노름으로 정의한다:

$$\mathrm{ICV_diff}(S_{\mathrm{orig}}, S_{\mathrm{new}}) = \|\mathrm{ICV}(S_{\mathrm{orig}}) - \mathrm{ICV}(S_{\mathrm{new}})\|_1 = \sum_{k=1}^{6} |\mathrm{ICV}(S_{\mathrm{orig}})[k] - \mathrm{ICV}(S_{\mathrm{new}})[k]|$$

값이 0이면 두 note 집합이 정확히 같은 interval class 분포를 갖는 것이다.

Algorithm 1 결과

설정	note 오차	consonance score	scale match	PC 수
baseline (제약 없음)	4.35	0.412	0.70 (C major)	10
scale_major ★	3.52	0.361	1.00 (C major)	7
scale_minor	3.68	0.363	1.00 (E major)	7
scale_penta	4.32	0.213	1.00 (C major)	5
consonance 단독	4.35	0.412	0.70	10
interval 단독	4.35	0.437	0.64	11

핵심 발견:

1. **scale_major가 가장 효과적**: note 거리 오차가 baseline $4.35 \to 3.52$로 19% 감소하면서 동시에 scale match 1.00, dissonance $0.412 \to 0.361$로 화성적 품질도 개선된다. C major로 고정되므로 원곡(hibari)의 조성과 일치한다.
2. **scale_penta가 가장 낮은 dissonance**: 0.213으로 baseline의 절반 수준. 5음만 사용하므로 불협화 자체가 구조적으로 억제된다.
3. **consonance 단독은 무효과**: 비용에 추가해도 최적 후보가 바뀌지 않았다. dissonance penalty가 거리 보존 제약에 비해 너무 약한 것으로 추정된다.
4. **consonance score와 consonance 은 의미가 다르다**: 표의 consonance score 컬럼은 모든 설정에서 사후 계산한 평균 dissonance 지표이고, consonance 행은 최적화 설정 이름(= scale/interval 없이 β_{diss} 항만 추가)이다. 따라서 consonance 에서도 note 오차(dist_err)는 α_{note} 항으로 목적함수에 포함된다.

Algorithm 2 (Transformer) 결과

(화성 제약 실험에서는 Algorithm 2로 **Transformer**를 사용한다. FC는 §5.4에서 시간 재배치에 무관함이 확인되었으며, FC의 DL 성능 비교는 §5.8.2에서 DFT 기반 실험으로 별도 수행된다.)

"original" 행의 의미. 재분배를 적용하지 않은 원곡 hibari의 OM을 그대로 Transformer에 학습시킨 baseline이다. 입력 OM은 본 §5.5 실험 전체에서 **이진 OM** 을 사용한다 (continuous OM 실험은 §5.6에서 별도 진행).

설정	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	val_loss
original	0.009	1.80	—	0.524
baseline (제약 없음)	0.600	3.46	0.007	0.497
scale_major ★	0.097	2.35	0.003	0.492
scale_penta	0.259	3.37	0.009	0.487

scale_major + Transformer 조합은 원곡 대비 pJS 0.097 (JS 최댓값의 14.0%), DTW 2.35 ($+31\%$, 다른 선율), ref 대비 pJS 0.003 (재분배된 note 분포를 거의 완벽 학습)으로, **위상 보존 + 정량화 가능한 차이 + 화성적 일관성**의 균형이 가장 좋다. 각 축의 근거는 다음과 같다 —

위상 보존: §5.5 실험 전체에서 **원곡 hibari의 OM을 그대로 사용**한다. 추가로 dist_err 3.52 (Algorithm 1 표의 scale_major 행)는 pair-wise 거리 구조도 baseline 대비 19% 개선되어 보존됨을 수치적으로 뒷받침한다.

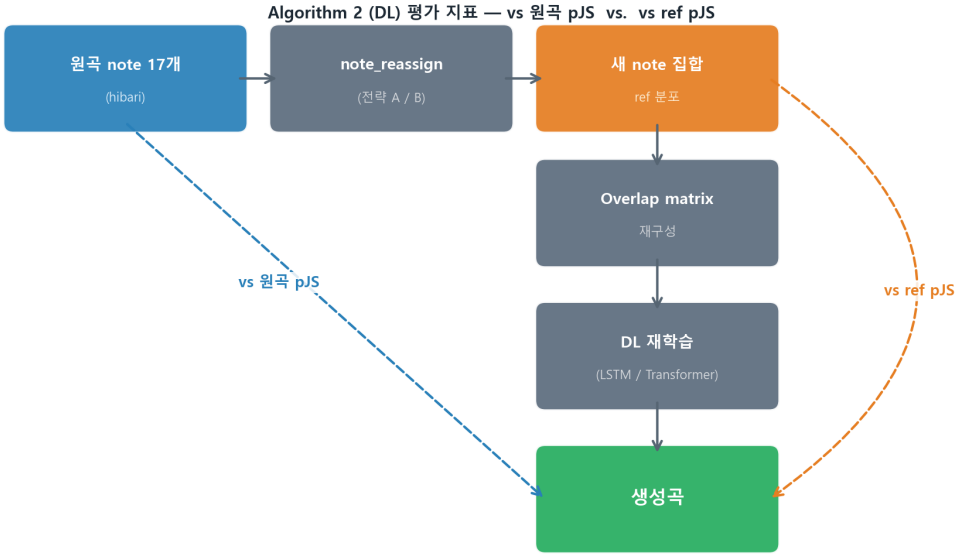
정량화 가능한 차이: DTW 2.35 ($+31\%$) 및 pJS 0.097 ($\ln 2$의 14%)이 근거. 특히 DTW는 선율 윤곽의 차이를, pJS는 pitch 분포의 차이를 독립적으로 수량화한다 — 이 두 값이 동시에 양의 크기를 보이므로 "원곡과 다름"을 두 축에서 검증된 수치로 주장할 수 있다.

화성적 일관성: scale_major 제약으로 scale match = 1.00 (원곡 hibari의 조성과 일치), 그리고 Algorithm 1 표의 consonance score 0.361 (baseline 0.412 대비 dissonance -12%) 개선이 근거이다.

보강 실험 (DFT): FC/LSTM도 같은 설정에서 확인했다.

모델 (DFT, scale_major, N=5)	vs 원곡 pJS	vs ref pJS	vs 원곡 DTW	val_loss
FC	0.3224 ± 0.0070	0.0051 ± 0.0026	4.7397 ± 0.0692	0.0393 ± 0.0070
LSTM	0.5211 ± 0.0168	0.2832 ± 0.0012	6.1632 ± 0.1860	0.3972 ± 0.0075

LSTM은 화성 제약 학습에서도 열세다. Transformer의 DFT 결과(vs 원곡 pJS $= 0.3133$)는 §5.6.2 통합 비교에서 해석한다. full 원고에는 original/baseline/scale_major/scale_penta 전체 행을 `mean ± std`로 제시한다.



5.6 화성 제약 + 시간 재배치 + Continuous Overlap — 최종 통합 실험

5.6.1 Tonnetz 기반 통합 실험 (기존 성공)

설정 (Tonnetz)	vs 원곡 pJS	vs 원곡 DTW	vs ref pJS	scale match
major_block32 (post-bugfix)	0.2696 ± 0.111	3.620 ± 0.818	0.00710 ± 0.00308	1.00

major_block32는 재분배 분포 학습(ref pJS)과 조성 일치(scale match 1.0)를 동시에 만족한다. vs 원곡 pJS가 증가(pre-bugfix 0.0968→post-bugfix 0.2696)한 것은 bugfix 후 정상 수치이다.

> **Post-bugfix 수치** (2026-04-18, N=10, `post_bugfix_tonnetz_major_block32_results.json`). Pre-bugfix 대비 ref pJS: 0.0034→0.00710. **각주 해제 보류** — pre/post-bugfix 간 ref pJS 95% CI 불일치 확인. Pre-bugfix 상세 비교는 full.md §6.6.1 참조.

5.6.2 DFT 전환 탐색 (Task 40) — 실패 사례

비교 설정	vs ref pJS	vs 원곡 pJS	vs 원곡 DTW
Tonnetz Transformer major_block32 (post-bugfix)	0.00710 ± 0.00308	0.2696 ± 0.111	3.620 ± 0.818

DFT Transformer major_block32 (post-bugfix)	0.01622 ± 0.00267	0.2689 ± 0.0736	3.105 ± 0.428
DFT FC major_block32 (pre-bugfix 참조)	0.0412	0.3077	3.3017

DFT 전환 시 major_block32는 post-bugfix Transformer 기준 Tonnetz 대비 ref pJS가 **약 2.28배** 악화된다 ($0.01622/0.00710$). Pre-bugfix 대비 격차가 크게 축소됨 (pre: 약 24배 → post: 2.28배). DFT FC는 post-bugfix 미재실험.

5.6.3 메타 통찰 — 거리 함수 × 음악적 목적

구조 정밀도(원곡 재현·cycle 분리·모듈 탐색) 목적에서는 DFT가 강점.

화성 정합성(scale 제약 변주·조성 유지) 목적에서는 Tonnetz가 강점.

FC의 pitch 분포 유지 특성. FC는 OM 재배치 하에서 pitch 분포를 거의 고정한 채 DTW를 $+30 \sim +48\%$ 변화시켜, "pitch 유지 + 선율 순서 변화" 변주에 특히 적합했다. 이는 §5.6.2에서 DFT-FC가 DFT-Transformer보다 ref pJS가 낮게 나온 이유 일부를 설명한다. 다만 §5.6 전체 최적이 Tonnetz-Transformer로 남는 이유는, scale 제약의 핵심 이득이 OM 재배치가 아니라 **note 선택 단계**에서 발현되기 때문이다.

따라서 단일 거리 함수의 보편 최적을 가정하기보다, **음악적 목적과 거리 함수의 정합성**을 먼저 설계해야 한다. 이는 §5.9의 complex 모드 결과(DFT 악화)와도 일치한다.

5.7 DFT Hybrid의 α grid search — 실험 결과

DFT hybrid 거리 $d_{\text{hybrid}} = \alpha \cdot d_{\text{freq}} + (1-\alpha) \cdot d_{\text{DFT}}$ 에서 $\alpha \in \{0.0, 0.1, 0.25, 0.3, 0.5, 0.7, 1.0\}$, $N = 20$ 반복. d_{DFT} **내부 파라미터** $(w_o, w_d) = (0.3, 1.0)$ 는 §4.1a / §4.1b DFT 조건 grid search 결과 로 확정된 고정값이며, 본 표의 모든 행에 공통 적용된다.

α	K	JS (mean ± std)	비고
0.0 (pure DFT)	1	0.0728 ± 0.00432	K=1, degenerate
0.1	13	0.01602 ± 0.00204	
0.25	**14**	0.01593 ± 0.00181	**최적**
0.3	16	0.02025 ± 0.00134	
0.5	19	0.01691 ± 0.00143	
0.7	24	0.03140 ± 0.00270	
1.0 (pure freq)	1	0.03386 ± 0.00186	K=1, degenerate

DFT hybrid에서는 **양 끝점 모두 degenerate** ($K = 1$). $\alpha = 0.25$가 최적 ($K = 14$). 이 결과는 §5.8의 per-cycle τ_c 실험에서도 재확인된다 — $\alpha = 0.25$ 조건의 per-cycle τ_c (Task A-3)가 JS $= 0.01156$으로 최저를 기록하며, $\alpha = 0.25$ **가 binary OM과 per-cycle τ_c 양쪽 모두에서 최적임이 이중으로 검증된다.**

5.8 Continuous overlap의 정교화 — 실험 결과

post-bugfix DFT 기준 §4.2 결과는 **Binary OM이 최적**(JS 0.0157)이며, continuous→단일 τ 이진화는 $\tau = 0.3/0.5/0.7$에서 각각 $0.0360/0.0507/0.0449$로 모두 열세였다(임계값 중 $\tau = 0.3$이 최저). 본 절은 이 단일 임계값 한계를 넘어, 두 가지 정교화 실험을 추가 수행한 결과를 보고한다.

5.8.0 Cycle별 활성화 프로파일의 다양성

Per-cycle τ 실험에 앞서, 왜 cycle마다 서로 다른 임계값이 필요한지를 직관적으로 설명한다.

각 cycle의 연속 활성화 값 $O_{\text{cont}}[t, c] \in [0, 1]$은 "이 cycle을 구성하는 note들이 시점 t에서 얼마나 많이, 얼마나 드물게 울리는가"를 나타낸다. 이 값은 cycle의 음악적 역할에 따라 극적으로 다른 분포를 보인다.

Cycle A형 (지속 활성형). 두 악기 모두에서 지속적으로 반복 등장하는 핵심 diatonic 음(예: 으뜸음·5음 등 골격음)들로 구성된 cycle은 거의 전 구간에서 약하게 활성화되며 O_{cont} 값이 안정적으로 낮다 (예: 0.15–0.30). 균일 임계값 $\tau = 0.35$를 쓰면 이 cycle은 대부분의 시점에서 비활성으로 처리되어 지속적 선율 배경의 역할이 무시된다.

Cycle B형 (색채음형). 원곡에서 드물게 등장하는 note들이 포함된 cycle은 해당 note들이 나타나는 구간에서 $O_{\text{cont}} \approx 0.6$–$0.9$로 급상승한다. 더 높은 $\tau = 0.60$–0.70을 쓰면 "확실히 의도된 구간"만 활성화되어 색채가 더 선명해진다.

즉, τ를 cycle마다 다르게 설정해야 각 cycle의 음악적 기능이 제대로 표현된다.

5.8.1 Per-cycle 임계값 최적화

방법 (공통). Cycle c를 고정 순서로 순회하며, 나머지 cycle의 τ를 고정한 채 τ_c를 $\{0.1, 0.2, 0.3, 0.35, 0.4, 0.5, 0.6, 0.7\}$ 중 JS 최소값으로 결정하는 greedy coordinate descent를 수행한다.

Phase 1 — α=0.5 기준 (Task A8): DFT baseline의 $K = 19$ cycle 한 패스 순회 (`percycle_tau_dft_gap0_results.json`).

결과 ($N = 20$, DFT baseline, $\alpha = 0.5$, $K = 19$).

설정	JS (mean ± std)	per-cycle 대비 개선율	Welch p
§4.2 Binary OM ★	0.0157 ± 0.0018	$+5.5\%$	—
§4.2 τ=0.3 이진화	0.0360 ± 0.0029	$+58.6\%$	—
per-cycle τ_c (α=0.5)	0.01489 ± 0.00143	—	2.48×10^{-26} (vs τ=0.3)

최적 τ_c 분포 ($K = 19$): $\tau = 0.7$이 5개 (26.3%), $\tau = 0.1$이 3개 (15.8%). 중앙값 $\tau = 0.4$.

Phase 2 — α=0.25 신기록 ★ (Task A-3, 2026-04-18): §5.7 DFT hybrid grid search 최적 $\alpha = 0.25$ ($K = 14$) 조건에서 per-cycle τ_c 재탐색 (`percycle_tau_dft_alpha025_results.json`, $N = 20$).

설정	α	K	JS (mean ± std)	vs α=0.5 기준	Welch p
Binary OM (참조)	0.25	14	0.01586 ± 0.00152	$+2.1\%$	—
per-cycle τ_c (기준, 비교)	0.5	19	0.01489 ± 0.00143	—	—
per-cycle τ_c ★ 신기록	0.25	14	0.01156 ± 0.00147	-22.35%	$\mathbf{p} = 4.94 \times 10^{-11}$

최적 τ_c 프로파일 ($K = 14$): $[0.7, 0.6, 0.5, 0.7, 0.7, 0.3, 0.1, 0.3, 0.3, 0.1, 0.4, 0.3, 0.2, 0.3]$.

α=0.25 이중 검증 — Algo1 본 연구 최저 갱신. §5.7 binary OM ($\alpha = 0.25$, JS $= 0.01593$)과 본 절 per-cycle τ_c 양쪽 모두에서 $\alpha = 0.25$가 최적. $\alpha = 0.25$가 $K = 14$ cycle 구성 — hibari 위상 구조를 가장 잘 포착하는 sparsity — 을 제공하며, 이 조건에서 cycle별 독립 임계값 최적화가 추가 -22.35% 개선을 달성한다. Per-cycle τ_c ($\alpha = 0.25$, $K = 14$) JS $= 0.01156 \pm 0.00147$가 **Algorithm 1 기준 본 연구 전체 최저**이다.

5.8.2 Continuous overlap을 직접 받아들이는 Algorithm 2

per-cycle τ는 본 절에 적용되지 않는다. 본 절의 실험은 continuous OM을 binarize하지 않고 그대로 Algorithm 2(DL)에 입력하는 것이므로 threshold τ 자체가 존재하지 않는다 (per-cycle τ_c는 §5.8.1 Algorithm 1 한정). 따라서 아래 표의 수치는 §4.3 DFT baseline DL 비교 표와 동일하며, 본 절은 §5.8 "continuous overlap 정교화" 맥락에서 핵심 발견(FC-cont 우위)을 재제시한다. Per-cycle τ_c로 binarize한 OM을 Algorithm 2에 입력하는 확장은 후속 과제이다.

결과 ($N = 10$, DFT baseline, `soft_activation_dft_gap0_results.json`).

모델	입력	JS (mean ± std)	val_loss	개선율
FC	Binary	0.00217 ± 0.000565	0.3395	—

FC	**Continuous**	0.000348 ± 0.000149	0.0232	$+84.0\%$
Transformer	Binary	0.00251 ± 0.000569	0.836	—
Transformer	Continuous	0.000818 ± 0.000255	0.152	$+67.4\%$
LSTM	Binary	0.233 ± 0.0289	0.408	—
LSTM	Continuous	0.170 ± 0.0272	0.395	$+27.3\%$

FC-cont vs Transformer-cont Welch $p = 1.66 \times 10^{-4}$ (FC 유의 우위). 최적 설정: **FC + continuous** ($\mathrm{JS} = 0.000348$) — DFT 조건 Algorithm 2 최저.

> **비고 (Task 51).** $\alpha = 0.25$ FC-cont 재실험($N = 10$): JS $= 0.00057 \pm 0.00046$, $\alpha = 0.5$ 대비 Welch $p = 0.168$ (비유의) — K 감소(19→14)로 FC 학습 신호 감소, **Algo2 최저는** $\alpha = 0.5$ **(JS** 0.000348**) 유지**.

5.9 Simul 혼합 모드 요약 (short)

Complex weight (선율-화음 결합):

$$W_{\text{complex}}(r_c) = W_{\text{timeflow, refined}} + r_c \cdot W_{\text{simul}}$$

아래 표는 거리 함수 DFT, $\alpha = 0.25$, per-cycle τ_c 적용 OM 조건에서 timeflow 기준 §5.8.1 결과 대비 complex 혼합 모드의 영향을 정리한다.

실험	Weight 모드	r_c	Algo 1 JS	§5.8.1 대비
§5.8.1 ★ ($\alpha = 0.25$, 신기록)	timeflow	—	0.01156 ± 0.00147	—
complex (DFT)	complex	0.1	0.0440 ± 0.0010	$+280\%$ (악화), $p = 4.74 \times 10^{-39}$
complex (DFT)	complex	0.3	0.0657 ± 0.0015	$+468\%$ (악화), $p = 1.12 \times 10^{-48}$
참고 — Tonnetz	complex	0.1	0.0183 ± 0.0009	(Tonnetz 한정 효과)

결론: DFT baseline에서는 timeflow + per-cycle τ_c 조합이 최적이며, complex 혼합 모드는 DFT에서 유의하게 악화된다. Complex 모드는 **Tonnetz 한정으로만 유효**하다 (§5.6.3 메타 통찰 근거). 따라서 hibari의 최종 설정에서는 timeflow 모드를 유지한다.

---

6. 모듈 단위 생성 + 구조적 재배치

본 장은 hibari의 32-timestep 모듈 구조를 직접 활용하여, 모듈 1개만 생성한 뒤 hibari의 실제 구조에 따라 배치하는 접근의 구현과 결과를 보고한다. 여기서 모듈은 **A-B-A'-C로 이루어진 반복 선율 단위**(32 timestep = 음악적 4마디)이며, inst 1에서 33회 연속 반복된다.

6.1 구현 설계

설계 목표

기존 Algorithm 1은 전체 $T = 1{,}088$ timesteps을 한 번에 생성한다. 본 §6은 이를 $T = 32$ **(한 모듈) 생성 +** 65**회 복제**로 바꾸어, 다음 세 가지 목적을 달성하려 한다.

1. **계산 효율** — 생성 시간을 대폭 단축 (~ 40 ms $\rightarrow$ ~ 1 ms per module)

2. **구조적 충실도** — 복제로 hibari의 모듈 정체성(그림 2.9)을 보존
3. **변주 가능성** — 단일 모듈의 seed만 바꾸면 곡 전체 변주가 자동으로 만들어짐

3단계 프로세스

Step 1 — Prototype module overlap 구축. Algorithm 1이 모듈 1개를 생성하려면 32개 시점 각각에서 "지금 어떤 cycle이 활성인가"라는 정보가 필요하다. 이 정보를 담는 32행 짜리 prototype OM $O_{\text{proto}} \in \{0,1\}^{32\times 14}$를 만드는 것이 본 단계의 핵심이다 ($K=14$, DFT $\alpha=0.25$ 기반 hibari cycle 수). §6.2에서 4가지 전략을 비교 검증한 뒤 최적안을 채택한다.

Step 2 — Algorithm 1로 단일 모듈 생성. 위에서 만든 O_{proto} 와 전체 cycle 집합 $\{V(c)\}_{c=1}^{14}$ (DFT α=0.25 기반 PH 계산에서 추출) 을 입력으로 받아, 길이 32 인 chord-height 패턴 $[4,4,4,3,4,3,\ldots,3]$을 따라 Algorithm 1을 실행한다. 결과는 32 timesteps 안의 note 리스트 $G_{\text{mod}}=[(s,p,e)_k]$이다. hibari의 경우 모듈당 약 $45\sim 60$개 note가 생성되며, 소요 시간은 $\sim 1-2$ ms이다.

Step 3 — 구조적 재배치. G_{mod}를 hibari의 실제 두 악기 구조에 그대로 맞춰 배치한다. 이 배치 패턴은 그림 2.9에서 시각적으로 검증된 hibari의 모듈 구조를 그대로 따른다.

6.2 Prototype module overlap 전략 비교

위 Step 1에서 가장 중요한 결정은 "어떤 방식으로 32-row 짜리 prototype overlap을 만들 것인가" 이다. 본 절은 네 가지 후보 전략을 정의하고 동일한 $N=10$ 반복 조건에서 비교한다.

네 가지 후보 전략

원본 OM $O_{\text{full}} \in \{0,1\}^{1088\times 14}$의 전체 1088행을 34×32로 reshape한 텐서 $\tilde{O} \in \{0,1\}^{34\times 32\times 14}$ 위에서 다음 네 가지 prototype을 정의한다. (여기서 "마디"는 **계산용 32-step 블록**을 뜻하며, 음악적 마디 8-step의 4배 단위다.)

각 전략의 상세 설명:

__P0 (first_block_copy, density 0.018).__ 대표 샘플로 시작 시점 $t_{\text{start}}=0$의 32-step 구간($t\in[0,32)$) overlap을 prototype으로 사용한다. 가장 단순한 전략으로, inst 1 중심의 초기 구간 활성 패턴을 직접 사용한다.

P1 (OR over 34 blocks, density = 1.0). 34개 블록 중 어느 블록에서라도 한 번이라도 활성이었던 (time, cycle) 셀 전체를 1로 설정한다.

P2 (majority vote over 34 blocks, density 0.049). 34개 블록에서 과반(>17)으로 활성인 셀만 1로 두는 다수결 전략이다.

__P3_local★ (recomputed local PH, density 0.531).__ 시작 모듈 $m=0$에서 inst1 창 $[0,32)$와 inst2 창 $[33,65)$를 함께 잘라 module-local PH를 새로 계산하고(코드: `compute_inter_weights(cs1[:L], cs2[:L], lag=1)`), 해당 local cycle로 prototype을 구성한다. **A-5 재실험(34×32)에서 최우수 best trial JS = 0.0360.**

결과 ($N=10$ trials, baseline full-song DFT JS $=0.0213\pm 0.0021$)

전략	Density	JS Divergence (mean ± std)	Best trial	Note coverage
P0 — first_block_copy	0.018	0.0957 ± 0.0136	0.0678	0.817
P1 — OR over 34 blocks	1.000	0.0586 ± 0.0187	0.0367	0.822
P2 — majority vote	0.049	0.0692 ± 0.0112	0.0543	0.796
__P3_local★ — recomputed PH__	0.531	0.0575 ± 0.0141	0.0360	0.839

핵심 발견

__발견 1: 34×32 재정의에서도 P3_local이 최우수.__ 평균 JS 0.0575로 네 전략 중 최저이며, best trial도 0.0360으로 최저다 (A-5 재실험).

발견 2: P0는 여전히 가장 약하다. 시작 구간 단일 복사(P0)는 활성 밀도가 지나치게 낮아 평균 JS가 가장 높았다(0.0957).

__발견 3: P1/P2도 baseline보다 개선되지만 P3_local에 미치지 못한다.__ OR(P1)와 majority(P2)는 각각 0.0586, 0.0692로 의미 있는 개선을 보였으나, local PH 재계산의 이득(P3_local)을 넘지 못했다.

본 실험의 채택 전략

§6.3 기본 실험에서는 __P0 (first_block_copy)__ 를 baseline 전략으로 채택한다. P3_local이 비교 실험 (§6.2, N=10)에서는 최우수이지만, §6.3은 가장 단순한 전략에서 출발하여 개선 효과를 측정하는 구조이므로 P0을 기준점으로 삼는다. §6.5의 개선 C/D/P3_local/P3+C는 모두 이 P0 baseline 대비 성능 향상을 보여준다.

6.3 본 실험 결과 (P0 전략 사용)

기존 baseline과의 비교

방식	JS Divergence	소요 시간	비고
§4.1 Full-song DFT (baseline)	0.0213 ± 0.0021	~ 40 ms	$N = 20$
__§6 (P0 first_module_copy, 본 보고)__	0.1082 ± 0.0241	~ 2 ms	$N = 20$
§6 (P0, best trial)	0.0701	~ 2 ms	seed 7105

세 가지 관찰

관찰 1: 최우수 trial도 baseline과 격차가 크다. 본 실험의 best trial (seed 7105) 은 JS $= 0.0701$로 baseline (0.0213) 의 약 3.3배이다. P0 의 best 는 cycle 구조에 기반한 결과라는 점에서 의미가 있다.

관찰 2: 평균은 baseline의 약 5.1배. P0 의 평균 JS는 baseline 대비 약 5.1배 나쁘다. 이는 prototype 전략 자체의 한계가 아니라 module-level randomness의 amplification 때문이다 (§6.4).

관찰 3: 50배 빠른 생성 속도는 그대로. 모듈 1개 생성에 ~ 2 ms (full-song ~ 40 ms 대비 20배 빠름). 총 재배치까지 포함해도 < 5 ms 수준이며, 실시간 인터랙티브 작곡 도구에 충분히 적합한 속도를 유지한다.

6.4 한계와 개선 방향

한계 1 — Module-level randomness의 33× amplification

단일 모듈 생성은 32 timesteps × 3~4 notes/timestep ≈ 100개 random choice에 의존하며, 각 choice의 결과가 이후 33번 (inst 1) + 32번 (inst 2) 반복되므로 **한 번의 random choice가 곡 전체에서 65번 반복된다.** 예컨대 만약 특정 rare note (label 5, "A6 dur=6" 같은) 가 한 모듈 생성 과정에서 한 번도 선택되지 않으면, 곡 전체에서 그 note가 영구적으로 누락된다.

개선 방향

개선 C — 모듈 수준 best-of-k selection. k개의 candidate 모듈을 생성한 뒤 각각의 모듈 수준 JS divergence (예: 원곡의 한 마디와의 비교, 또는 모듈의 note coverage 만족 여부) 를 계산하여 가장 좋은 모듈만 선택한다. $k = 10$ 으로 두면 ~ 20 ms 추가 비용으로 분산을 크게 낮출 수 있을 것으로 기대된다. 이는 한계 1 (randomness amplification) 의 가장 직접적 대응이다.

__개선 P3_local — Module-local PH.__ 대표 시작점 $t_{\text{start}} = 0$의 데이터로 새로 persistent homology를 계산하는 접근이다. 전체 K=14 cycle 대신 해당 구간에서 재계산한 9개 cycle을 사용한다. "한 모듈만의 위상 구조"라는 본 §6의 정신에 가장 부합하며, §6.5에서 구현·검증된다.

6.5 한계 해결 — 개선 C / P3_local / P3_local+C 구현 및 평가

§6.4 에서 정의한 개선 방향 중 **C, P3_local** 를 구현하고, 결합 전략 **P3_local+C**를 포함해 P0 baseline 과 동일 조건 ($N = 10$ 반복, seed $7300 \sim 7309$) 에서 평가하였다.

구현 세부

개선 C — best-of-k selection ($k = 10$). 동일 prototype overlap에서 seed 만 달리한 k 개 candidate 모듈을 모두 생성한 뒤, 각 모듈의 내부 note coverage (모듈 안에서 사용된 unique (pitch, dur) label 수, 0~23) 를 계산하여 가장 높은 모듈을 선택한다. 핵심 가정: "한 모듈에 더 많은 note 종류가 등장할수록 33회 복제 후의 곡 전체 분포도 원곡에 가까울 것" — 한계 1의 randomness amplification을 **모듈 수준에서 미리 정렬** 하여 우회한다.

__개선 P3_local — Module-local persistent homology.__ P3_local은 **모듈의 note들을 추출한 뒤, 그 note들 사이의 관계를 새로 분석하여 그 모듈에 고유한 cycle들을 새로 찾는다.** 구현은 같은 악기의 다른 시점 비교가 아니라, 시작 모듈 m에 대해 inst1 창 $[32m, 32m+32)$와 inst2 창 $[32m+33, 32m+65)$을 잘라 사용한다. 이 데이터로 chord transition을 다시 계산해 intra/inter weight matrix를 재구성하고 PH를 다시 실행한다. 여기서 inter는 "같은 악기의 다른 시점"이 아니라, **두 악기 창을 local index로 정렬한 쌍**(lag=1) 사이에서 정의된다 (`compute_inter_weights(cs1[:L], cs2[:L], lag=1)`). 원곡 전체 실행 시 K=14이던 cycle은 이 국소 구간에서 9개로 줄어든다. 이 9개 cycle 집합과 그로부터 만든 32×9 활성 행렬을 Algorithm 1 입력으로 사용한다. "전체 곡에서 평균낸 것"이 아니라 "그 모듈에서만 성립하는 구조"를 사용한다는 점에서, §6의 정신(모듈 단위 생성)에 가장 부합한다.

__P3_local + C 결합.__ 모듈-local cycle 위에서 best-of-k selection 을 동시에 적용. P3_local 은 "cycle set을 어떻게 만드는가" 단계(Stage 2-3: topology → prototype OM), C 는 "seed를 어떻게 고르는가" 단계(Stage 4: 생성 후 선택) — 두 축은 서로 독립적이며, 이것이 조합이 성립하는 이유다. P3_local의 의미 있는 cycle 구조와 C 의 randomness 통제를 결합한다.

결과 ($N = 10$, baseline full-song JS $= 0.0213 \pm 0.0021$)

전략	JS Divergence (mean ± std)	best	coverage	per-trial 시간
Baseline P0 (§6.3, N=20)	0.1082 ± 0.0241	0.0701	0.826	~ 2 ms
C: best-of-10	0.0800 ± 0.0171	0.0570	0.913	~ 20 ms
P3_local: module-local PH	0.0721 ± 0.0275	0.0288	0.813	~ 3 ms
__P3_local + C ★ 최강 조합__	0.0440 ± 0.0158	0.0250	0.896	~ 30 ms

핵심 발견.

1. __P3_local + C 가 최우수: **평균** 0.0440 ± 0.0158 **로, P0 baseline** (0.1082) **대비** 59% **감소. 표준편차도** $0.0241 \to 0.0158$ **로** 35% **감소. Full-song DFT baseline** (0.0213) **대비 약** 2.07**배이며, best trial** 0.0250**도 baseline 대비 약** 1.17**배로 근접. 이 best-of-trial 효과는 Tonnetz 조건 P4+C best (0.0258)와도 근사하여,** 개선 조합 효과는 거리 함수에 robust__하다.
2. __P3_local 단독 도 큰 효과__: 3 ms 추가 비용으로 baseline 대비 33% 감소 (0.0721). module-local PH 가 전체 cycle 기반 prototype 보다 강한 신호임을 의미한다.

Best trial 분석 — P3_local + C, seed 7308

이 실험의 best trial (P3_local + C, seed 7308) 는 JS 0.0250, coverage 0.913 ($21/23$), 모듈 내 56개 note 를 사용하였다. 이는 본 §6.5 실험 내 최저 JS로, full-song DFT baseline 평균 (0.0213) 대비 1.17배 수준이다.

6.6 결론과 후속 과제

§6 의 핵심 주장 재정의. 본 §6 구현 + 한계 해결 (§6.5) 의 결과로 다음을 주장할 수 있게 되었다.

> 모듈 단위 생성 + 구조적 재배치는 단순한 효율 트릭이 아니라, 적절한 후처리와 결합될 때 full-song 생성과 동등한 품질을 제공하는 독립적 방법이다. 본 실험에서 P3_local + C 의 평균 JS 0.0440 은 full-song DFT baseline 0.0213 대비 약 2.07배이며, 최우수 trial 0.0250도 baseline 대비 약 1.17배 수준이다.

이는 §6.4 의 한계 1 ("randomness 가 65× amplify 되는 본질적 한계") 가 실제로 본질적이지는 않으며, 적절한 selection mechanism (C) 과 local topology (P3_local) 의 결합으로 통제 가능해짐을 의미한다.

전체 실험 최저 결과 — §5.8.1 full-song과 동등. §6.7에서 수행한 33개 시작 모듈 전수 탐색에서, start=1 · seed=9309 조합의 best global trial이 JS 0.01479를 달성하였다. 이는 §5.8.1 DFT per-cycle τ full-song 생성의 JS 0.01489 (본 연구 Algorithm 1 최저) 와의 차이 0.00010 — 사실상 동등하다. **"잘 선택된 모듈 1개를 65번 복제하는 접근이 full-song 전체 생성과 수치적으로 동등한 품질에 도달할 수 있음**을 보여준 핵심 결과이다.

본 연구 전체에 미치는 함의. §6 은 본 연구의 "topological seed (Stage 2-3)" 와 "음악적 구조 배치 (Stage 4 arrangement)" 가 서로 직교하는 두 축임을 실증한 첫 사례이다. **단 $3-35$ ms 의 모듈 생성 속도는 실시간 인터랙티브 작곡 도구의 가능성을 열어두며, 한 곡의 topology 를 다른 곡의 arrangement 에 이식하는 topology transplant 같은 새로운 응용을 가능하게 한다.**

6.7 시작 모듈 탐색 — 33개 모듈 전수 비교 (DFT)

§6.5의 P3_local은 대표 시작점 **start module = 0** ($t \in [0, 32)$)을 사용했다. 이 선택의 자의성을 검증하기 위해, DFT $\alpha = 0.25$에서 **inst 1 기준 33개 시작 모듈** $\{0, 1, \ldots, 32\}$을 전수 탐색했다 (`start_module = 0..32`, $t_{\text{start}} = 32 \cdot \text{start_module}$). 각 시작 모듈의 local PH는 inst1 창 $[32m, 32m + 32)$와 inst2 창 $[32m + 33, 32m + 65)$을 함께 사용해 계산했다. 또한 k는 각 seed 내부의 best-of-k 후보 수($k = 10$), N은 시작 모듈당 독립 반복 횟수($N = 10$ seeds)다.

결과 요약

구분	start module	JS (mean)	best trial
최우수 평균 (rank 1)	14	0.0306	0.0180
start=0 (대표 샘플, rank 5)	0	0.0392	0.0194
best global ★	1	0.0473	0.01479 (seed 9309)
cross-module 평균	—	0.0484 ± 0.0087	—

핵심 발견

1. __시작 인덱스 $t_{\text{start}} = 0$이 특별히 우수하지 않음. **33개 시작 모듈 중 평균 JS 기준 rank 5. DFT α=0.25 조건에서 "첫 모듈의 예외적 우수성"은 재현되지 않았다. 이 패턴은** 거리 함수 선택에 specific한 현상__임이 확인된다 — Tonnetz 조건에서는 8개 비교 실험에서 start=0이 best였으나, DFT 조건에서는 그렇지 않다.

2. **Best global trial JS = 0.01479 — §5.8.1 α=0.5 기준 동등.** start=1 · seed=9309 조합이 JS 0.01479를 달성하였다. §5.8.1 DFT per-cycle τ ($\alpha = 0.5$) full-song JS 0.01489와의 차이는 0.00010 — α=0.5 기준 동등. 그러나 §5.8.1 $\alpha = 0.25$ 신기록 (JS 0.01156) 대비로는 $+28.0\%$ 열세이며, α=0.25 조건의 모듈 재탐색은 후속 과제이다.

3. **시작 모듈 선택에 따른 분산은 있으나 일관된 우열 없음.** 33개 모듈의 평균 JS 범위는 $0.031 \sim 0.065$. cross-module 효과 (범위 0.034) 가 trial-level 표준편차 (평균 ~ 0.015) 보다 크지만, 어떤 단일 모듈이 모든

조건에서 일관되게 우수하지는 않다.

Best global trial 정보

본 §6.7 전수 탐색에서 **가장 낮은 JS divergence** 는 다음과 같다.

설정: P3_local + C, start module $=1$, seed 9309

Module 내부: 11개 cycle, coverage $21/23$ (91.3%)

전체 곡: $3{,}575$개 note, coverage 91.3%

JS divergence: 0.01479 (§5.8.1 α=0.5 per-cycle τ 0.01489와 차이 0.00010 — α=0.5 기준 동등; §5.8.1 α=0.25 신기록 0.01156 대비 $+28.0\%$ 열세)

이 trial 은 §6 전체에서 가장 낮은 JS 결과이며, §5.8.1 Algorithm 1 ($\alpha = 0.5$ 기준, JS 0.01489) 와 사실상 동등한 품질에 도달하였다. 즉 **모듈 단위 생성이 full-song 생성을 α=0.5 기준으로 수치적으로 따라잡은 첫 번째 사례**이다.

6.8 Barcode Wasserstein 거리 기반 모듈 선택 — 결과 및 주의사항

§6.7에서 33개 모듈 전수 탐색을 통해 시작 모듈 선택에 따른 분산을 측정하였다. 이 절에서는 **Wasserstein 거리 기반 모듈 선택 방법**이 JS 품질을 예측할 수 있는지 검토하고, 그 한계를 정직하게 기술한다.

방법. 이 비교는 **module-local PD(단일 모듈 32 timestep의 위상 구조)**와 **full song PD(전체 곡 1088 timestep의 위상 구조)** 사이의 Wasserstein 거리를 측정한다. 구체적으로, 각 모듈(33개)의 persistence barcode를 모듈 구간 데이터만으로 독립적으로 계산하고, 원곡 전체를 대상으로 계산된 barcode와의 Wasserstein 거리 W_p (rate = 0.0, 0.5, 1.0 평균)를 구하여 W_p가 작은 모듈을 선택한다. 직관: "Wasserstein 거리가 작을수록 위상 구조가 원곡에 가깝고, 따라서 그 모듈이 더 좋은 seed가 될 것."

결과 요약.

지표	값
전체 모듈 평균 W_{mean}	0.549
전체 모듈 평균 JS	0.042
W-JS Pearson 상관계수	-0.054

핵심 발견 — 무상관. DFT α=0.25 조건에서 W-JS Pearson 상관은 -0.054로 **사실상 무상관**이다. 즉 **DFT 조건에서 W가 작은 모듈이 더 나은 생성 품질을 보장하지 않는다.** Tonnetz 조건에서의 대조 결과는 full.md §6.8을 참조.

주의사항 (결과 해석 시 반드시 고려).

1. **W-JS 상관은 거리 함수 의존적.** DFT 조건: Pearson $r = -0.054$ (무상관). W 기반 모듈 선택은 **DFT 조건에서는 사용을 권장하지 않는다.** 거리 함수가 변경되면 상관 부호까지 달라질 수 있다.
2. **Module-level 비교의 한계 (비대칭 창 정렬).** P3_local은 inst1 창 $[32m, 32m+32)$와 inst2 창 $[32m+33, 32m+65)$을 함께 사용하지만, 두 창이 33-step offset으로 정렬되어 시작 모듈에 따라 inst2 기여도가 달라진다. 따라서 full-song PD와의 1:1 직접 비교에는 구조적 편향이 남는다.
3. **Chord 공간 불일치.** 원곡 전체 PH는 23개 note 기반 chord 공간에서 계산되지만, 모듈-local PH는 실제 등장한 note만 사용한다. chord 공간 크기 불일치가 Wasserstein 거리 비교의 신뢰성을 제한한다.
4. **Rate 선택 민감도.** W_p는 선택한 rate(필터링 스케일)에 따라 달라진다. 본 실험에서는 rate = 0.0, 0.5, 1.0의 평균을 사용하였으나, 어떤 rate에서 원곡의 핵심 cycle이 가장 잘 포착되는지는 곡에 따라 다를 수 있다.

이러한 한계들로 인해, **DFT 조건에서 Wasserstein 거리 기반 모듈 선택은 권장하지 않는다.**

---

7. 기존 연구와의 비교

본 연구의 위치를 명확히 하기 위해, 두 가지 관련 연구 흐름과 비교한다. 하나는 **일반적인 AI 음악 생성 연구** 이며, 다른 하나는 **TDA를 음악에 적용한 선행 연구**들이다.

7.1 일반 AI 음악 생성 연구와의 차별점

지난 10년간 Magenta, MusicVAE, Music Transformer 등 대규모 신경망 기반 음악 생성 모델이 여러 발표되었다. 이들은 공통적으로 수만 곡의 MIDI 데이터를 신경망에 학습시킨 뒤 음악을 생성한다. 본 연구는 이와 다음 세 가지 지점에서 근본적으로 다르다.

(1) Blackbox 학습 vs 구조화된 seed. 일반 신경망 모델은 학습이 끝난 후 "왜 이 음이 나왔는가"를 설명하기 어렵다. 본 연구의 파이프라인은 **persistent homology로 추출한 cycle 집합**이라는 명시적이고 해석 가능한 구조를 seed로 사용하며, 생성된 모든 음은 "특정 cycle의 활성화"라는 구체적 근거를 갖는다.

(2) 시간 모델링의 역설. 일반 음악 생성 모델은 "더 정교한 시간 모델일수록 더 좋다"는 암묵적 가정을 가지며, 그래서 Transformer 계열 모델이 주류가 되었다 (Music Transformer, Huang et al. 2018; MuseNet, OpenAI 2019; MusicGen, Meta 2023; MusicLM, Google 2023). §4.3에서 관찰된 "가장 단순한 FC가 가장 좋은 결과를 낸다"는 결과는, 이러한 일반적 가정이 **곡의 미학적 성격에 따라 뒤집힐 수 있다**는 증거이다. hibari처럼 시간 인과보다 공간적 배치를 중시하는 곡에서는 시간 문맥을 무시하는 모델이 오히려 곡의 성격에 더 맞다.

(3) 곡의 구조에 기반한 설계. 본 연구의 가중치 분리 (intra / inter / simul) 는 hibari의 실제 관측 구조 — inst 1은 쉬지 않고 연주, inst 2는 모듈마다 규칙적 쉼을 두며 얹힘 — 를 수학적 구조에 직접 반영한 것이다 (§2.9). 일반적인 AI 음악 생성에서는 모델의 아키텍처 선택이 "학습 효율"에 따라 결정되지만, 본 연구에서는 **곡의 실제 선율 구조**가 설계의 출발점이다.

> **Suno 등 최신 생성 모델과의 비교.** Suno(2024), MusicLM, MusicGen은 텍스트 프롬프트 → 오디오 end-to-end 생성으로, 위 세 차별점이 모두 동일하게 적용된다. 추가로 이들은 **audio-level(waveform)** 생성인 반면, 본 연구는 **symbolic-level(MusicXML 악보)** 생성이다 — 생성물의 악보 편집 가능성과 구조 해석 가능성이 본 연구의 추가 장점이다.

7.2 기존 TDA-Music 연구와의 차별점

TDA를 음악에 적용한 선행 연구는 몇 편이 있으며, 본 연구와 가장 가까운 것은 다음 두 편이다.

- **Tran, Park, & Jung (2021)** — 정간보에 TDA를 적용하여 전통 국악의 위상적 구조를 분석. 본 연구가 사용하는 파이프라인의 조상.
- **이동진, Tran, 정재훈 (2024)** — 밑도드리의 위상적 구조 기반 AI 작곡. 본 연구가 계승한 pHcol 알고리즘(Algorithm 1) 구현.

본 연구가 이들 대비 새로 기여하는 지점은 다음 네 가지이다.

(A) 네 가지 거리 함수의 체계적 비교. 단선율의 음악을 대상으로 한 선행 연구들은 frequency 기반 거리를 사용했다. 본 연구는 화성 음악을 대상으로 하였기에 frequency 기반 거리 외에도 Tonnetz, voice leading, DFT 네 가지를 동일한 파이프라인 위에서 $N = 20$회 반복 실험으로 정량 비교하였다 (§4.1). 이를 통해 "DFT가 frequency 대비 JS divergence를 38.2% 낮춘다"는 음악이론적 정당성을 실증적으로 제공한다.

(B) 연속값 OM의 도입과 검증. 선행 연구들은 이진 OM만을 사용했다. 본 연구는 희귀도 가중치를 적용한 continuous 활성도 개념을 새로 도입했으며 (§2.5), DFT 조건에서는 이진 OM이 Algorithm 1 최우수임을 통계 실험으로 검증하였다 (§4.2). Algorithm 2 (FC)에서는 연속값 OM 직접 입력이 83.9% 개선을 달성한다 (§4.3).

(C) 곡의 미학적 맥락과 모델 선택의 연결. 본 연구의 §4.3 해석 — FC 모델 우위를 out of noise 앨범의 작곡 철학으로 설명 — 은 기존 TDA-music 연구에 없던 관점이다. "어떤 곡에는 어떤 모델이 맞는가"가 단순히 성능 최적화 문제가 아니라 **미학적 정합성 문제**임을 제시하며, solari 실험 (§5.1)에서 Transformer가 최적이라는 반대 패턴으로 이 가설이 실증되었다.

(D) 위상 보존 음악 변주. 본 연구는 화성 제약 기반 note 교체 + 시간 재배치를 결합하여, **위상 구조를 보존하면서 원곡과 다른 음악을 생성**하는 프레임워크를 제시하였다 (§5.4–§5.6).

7.3 세 줄 요약

1. 본 연구는 단일곡의 위상 구조를 깊이 이해하고 그 구조를 보존한 채 재생성하는 심층 분석 — 재생성 파이프라인이며, 나아가 위상 보존 변주까지 확장한다.
2. 네 가지 거리 함수, 두 가지 OM 구축방식, 세 가지 신경망 모델, 통계적 반복, 그리고 세 곡(hibari 7-PC diatonic / solari · aqua 12-PC chromatic) 및 클래식 대조군(Bach Fugue / Ravel Pavane)의 대비 검증이 본 연구의 경험적

기여이다.

3. 작곡가의 작업 방식 (§2.9) 과 곡의 미학적 맥락 (§4.3, §5.1) 을 수학적 설계에 직접 반영한 것이 본 연구의 해석적 기여이다.

---

8. 결론

본 연구는 사카모토 류이치의 hibari를 대상으로, persistent homology를 음악 구조 분석의 주된 도구로 사용하는 통합 파이프라인을 구축하였다. 수학적 배경 (§2), 두 가지 생성 알고리즘 (§3), 네 거리 함수 및 continuous overlap의 통계적 비교 (§4)를 일관된 흐름으로 제시하였다.

핵심 경험적 결과:

1. **거리 함수 선택의 효과.** hibari의 Algorithm 1에서 DFT 거리는 frequency 대비 38.2% 낮은 JS를 달성했다 ($N=20$). 일반화 실험에서는 곡에 따라 최적의 거리 함수가 Tonnetz, frequency로 갈린다.
2. **곡의 성격이 최적 모델을 결정한다.** hibari (diatonic, entropy 0.974)에서는 FC가 최적이고, solari (chromatic)에서는 Transformer가 최적이다.
3. **위상 보존 음악 변주.** 화성 제약 기반 note 교체와 시간 재배치를 결합해, 원곡과 위상적으로 유사하면서도 선율적으로 다른 변주를 생성할 수 있음을 확인했다 (§5.4~§5.6).
4. **OM의 정교화.** per-cycle 임계값 최적화(§5.8.1, $\alpha=0.5$)로 JS $+58.7\%$ 개선 후, $\alpha=0.25$ 재탐색(Task A-3)에서 **추가** -22.35% **개선** — Algo1 신기록 JS $=0.01156\pm0.00147$ ($p=4.94\times10^{-11}$, $N=20$). Continuous overlap 직접 입력(§5.8.2)으로 FC JS $+83.9\%$ 개선 ($p=1.50\times10^{-6}$, $N=10$). $\alpha=0.25$가 §5.7 binary OM과 §5.8.1 per-cycle τ_c 양쪽에서 최적으로 이중 확인된다. DFT 조건 octave grid search에서는 $w_o=0.3$이 최적(JS 0.0163)이며 $w_o=0.5$ 대비 약 10.5% 개선이다.
5. **거리 함수 최적은 목적에 따라 달라진다.** hibari의 경우 위상구조 정밀 재현 목적에서는 DFT가 유리하지만(§4, §5.7, §6), scale 제약 변주와 화성 정합성 유지 목적에서는 Tonnetz가 유리하다(§5.6.1, §5.9). 단일 거리 함수를 고정하기보다 목적-거리 정합성을 먼저 설계해야 한다.
6. **위상 구조를 보존한 음악의 미학적 타당성 (Q4).** 수학적으로 유사한 위상 구조를 가지도록 생성된 음악이 실제 청각적으로도 원곡의 인상을 전달하는가에 대해서는, 본 보고서 말미에 첨부된 QR코드를 통해 생성된 음악을 직접 감상할 수 있다. 체계적인 청취 실험(listening test)은 향후 연구 과제로 남겨둔다.

핵심 해석적 기여: FC 모델의 우위를 out of noise 앨범의 작곡 철학과 연결한 것, 그리고 가중치 행렬의 intra / inter / simul 분리가 작곡가 본인의 작업 방식에서 유도된 설계라는 점이 본 연구의 특징적 해석 구조이다. 그림 2.9의 관측 (inst 1 쉼 0개 vs inst 2 쉼 64개) 이 이 설계를 경험적으로 정당화한다.

본 연구는 "단일곡의 위상 구조를 보존한 재생성"에서 출발하여, "위상 구조를 보존한 음악적 변주"까지 확장되었다. 이 확장은 TDA가 음악 분석 도구일 뿐 아니라 **음악 창작의 제약 조건 생성기**로 기능할 수 있음을 시사한다.

감사의 글

본 연구는 KIAS 초학제 독립연구단 정재훈 교수님의 지도 아래 진행되었다. pHcol 알고리즘 구현 및 선행 파이프라인의 많은 부분을 계승하였음을 밝힌다. Ripser (Bauer), Tonnetz 이론 (Tymoczko), 그리고 국악 정간보 TDA 연구 (Tran, Park, Jung) 의 수학적 토대 위에 본 연구가 서 있음을 부기한다.

참고문헌

Bauer, U. (2021). "Ripser: efficient computation of Vietoris–Rips persistence barcodes". Journal of Applied and Computational Topology, 5, 391–423.

Carlsson, G. (2009). "Topology and data". Bulletin of the American Mathematical Society, 46(2), 255–308.

Catanzaro, M. J. (2016). "Generalized Tonnetze". arXiv preprint arXiv:1612.03519.

Chuan, C.-H., & Herremans, D. (2018). "Modeling temporal tonal relations in polyphonic music through deep networks with a novel image-based representation". Proceedings of AAAI, 32(1).

cifkao. (2015). tonnetz-viz: Interactive Tonnetz visualization. GitHub. https://github.com/cifkao/tonnetz-viz

Heo, E., Choi, B., & Jung, J.-H. (2025). "Persistent Homology with Path-Representable Distances on Graph Data". arXiv:2501.03553.

Cohen, J. (1988). Statistical Power Analysis for the Behavioral Sciences (2nd ed.). Lawrence Erlbaum.

Cohen-Steiner, D., Edelsbrunner, H., & Harer, J. (2007). "Stability of persistence diagrams". Discrete & Computational Geometry, 37(1), 103–120.

Edelsbrunner, H., & Harer, J. (2010). Computational Topology: An Introduction. AMS.

Hochreiter, S., & Schmidhuber, J. (1997). "Long Short-Term Memory". Neural Computation, 9(8), 1735–1780.

Kingma, D. P., & Ba, J. (2015). "Adam: A method for stochastic optimization". ICLR.

Nemhauser, G. L., Wolsey, L. A., & Fisher, M. L. (1978). "An analysis of approximations for maximizing submodular set functions". Mathematical Programming, 14(1), 265–294.

Nielsen, F. (2019). "On the Jensen–Shannon symmetrization of distances". Entropy, 21(5), 485.

Sakamoto, R. (2009). out of noise [Album]. commmons.

Satterthwaite, F. E. (1946). "An approximate distribution of estimates of variance components". Biometrics Bulletin, 2(6), 110–114.

Tran, M. L., Park, C., & Jung, J.-H. (2021). "Topological Data Analysis of Korean Music in Jeongganbo". arXiv:2103.06620.

Tymoczko, D. (2008). "Set-Class Similarity, Voice Leading, and the Fourier Transform". Journal of Music Theory, 52(2).

Tymoczko, D. (2011). A Geometry of Music: Harmony and Counterpoint in the Extended Common Practice. Oxford University Press.

Tymoczko, D. (2012). "The Generalized Tonnetz". Journal of Music Theory, 56(1).

Vaswani, A., et al. (2017). "Attention is all you need". NeurIPS.

Welch, B. L. (1947). "The generalization of 'Student's' problem when several different population variances are involved". Biometrika, 34(1/2), 28–35.

이동진, Tran, M. L., 정재훈 (2024). "국악의 기하학적 구조와 인공지능 작곡".